



جامعة حلب
كلية الهندسة المعلوماتية
قسم البرمجيات
السنة الخامسة

منصة لإدارة شؤون المدينة الجامعية باعتماد منهجية (microServices - docker)

إعداد الطلاب :

محمد ابراهيم ابراهيم رضا الامير

نوار ابراهيم أبو عماش محمد كمال الغريب

عبد الغفور زكريا المحمد حسين محمد زلط

إشراف

الدكتورة : مروه دحدوح

يغطي هذا المشروع أحد متطلبات نيل الإجازة في الهندسة المعلوماتية

ملخص المشروع

تُقدّم هذه الأطروحة توثيقًا تقنيًا وأكاديميًا لمشروع «منصة سكني لإدارة شؤون المدينة الجامعية»، وهي منصة برمجية متكاملة تهدف إلى أتمتة وتبسيط العمليات المرتبطة بالسكن الجامعي والخدمات الطلابية، عبر تبني معمارية الخدمات المصغّرة (Microservices) وتقنية الحاويات (Docker).

يعتمد النظام على خمس خدمات مصغّرة مستقلة: خدمة المصادقة (Auth Service)، وخدمة السكن (Housing Service)، وخدمة طلبات الخدمة (Service Requests Service)، وخدمة الشكاوى والاقتراحات (Feedback Service)، وخدمة الإشعارات والإعلانات (Notifications & Announcements Service)، تتواصل جميعها عبر بوابة API موحّدة (Ocelot API Gateway).

يوثّق البحث المعمارية المعتمدة، ومخططات تحليل النظام (ERD و Use Case) على المستويين العام والخاص بكل خدمة، وقواعد البيانات المستخدمة، وواجهات المستخدم على الموبايل ولوحة تحكم الإدارة، إضافة إلى القرارات التصميمية والمبررات الهندسية وراءها.

الفهرس

2.....	ملخص المشروع
3.....	الفهرس
1.....	الفصل الأول: المقدمة
1.....	أهمية المشروع
1.....	وصف وتوضيح المشروع
1.....	خدمات المشروع
3.....	الفصل الثاني: الأدوات ولغات البرمجة المستخدمة
3.....	لغات وأطر العمل المستخدمة في تطوير قسم Back-end
5.....	لغات وأطر العمل المستخدمة في تطوير قسم Front-end
9.....	المحررات والتطبيقات المستخدمة في عمليات التطوير
12.....	الفصل الثالث: تحليل المشروع وتوثيق البيانات
12.....	ملاحظة منهجية حول استخدام ERD
12.....	مخطط ERD العام (المفاهيمي)
12.....	مخططات كل خدمة على حدة
13.....	أ. خدمة المصادقة Auth Service
14.....	ب. خدمة الشكاوى والاقتراحات Feedback Service
16.....	ج. خدمة الإشعارات والإعلانات Notifications & Announcements Service
18.....	د. خدمة السكن Housing Service
19.....	هـ. خدمة طلبات الخدمة Service Requests Service
21.....	جدول العلاقات بين الجداول
24.....	الفصل الرابع: عرض واجهات المشروع
24.....	أولاً: تطبيق الموبايل Flutter واجهة الطالب
41.....	ثانياً: لوحة تحكم الإدارة
49.....	المراجع References
50.....	English Abstract

الفصل الأول: المقدمة

الفصل الأول: المقدمة

أهمية المشروع

مع تزايد الاعتماد على التكنولوجيا في مختلف مناحي الحياة الجامعية، برزت الحاجة إلى رقمنة الخدمات المرتبطة بالسكن الجامعي بما يخفف الأعباء الإدارية عن الجهات المشرفة، ويحسن تجربة الطالب في التسجيل والمتابعة والتواصل مع الإدارة. تكمن أهمية المشروع في معالجته لثغرة حقيقية: غياب منصة موحدة تجمع إدارة السكن والخدمات والشكاوى والتواصل ضمن نظام رقمي واحد متكامل، بدلاً من الاعتماد على إجراءات ورقية أو قنوات تواصل غير رسمية ومتفرقة.

وصف وتوضيح المشروع

«سكني» منصة برمجية متكاملة تُدير شؤون المدينة الجامعية من طرفين رئيسيين: تطبيق موبايل (Flutter) موجّه للطالب، ولوحة تحكم ويب (React) موجّهة للإدارة. تعتمد المنصة معمارية الخدمات المصغرة (Microservices)، بحيث تمثل كل خدمة مجال عمل مستقل بمنطقة الخاص وقاعدة بياناته المستقلة، وتتواصل جميع الخدمات عبر بوابة API موحدة (Ocelot API Gateway) تستقبل طلبات كل من الموبايل والويب وتوجّهها للخدمة المناسبة.

اعتمد فريق المشروع في مرحلة التصميم منهجية تصميم موحدة (Design System) قائمة على اللون الأساسي #0EA5E9 وألوان دلالية للحالات (نجاح، تحذير، خطر)، وخط Cairo، وأيقونات Lucide بنمط Outline، مع مكونات واجهة ذات ظلال ناعمة وحواف دائرية أكبر وحالات تحميل هيكلية (Skeleton Loading)، بما يضمن اتساق تجربة المستخدم بين الموبايل ولوحة التحكم.

خدمات المشروع

- يتوزع منطق النظام على خمس خدمات مصغرة مستقلة، كل منها مسؤولة عن مجال عمل محدد:
- خدمة المصادقة (Auth Service): التسجيل، تسجيل الدخول، إدارة الهوية والأدوار، وإصدار رموز الدخول JWT.
 - خدمة السكن (Housing Service): إدارة الغرف والوحدات السكنية ومجموعات الطلاب وطلبات الحجز.
 - خدمة طلبات الخدمة (Service Requests Service): استقبال طلبات الطلاب المتعلقة بالخدمات التشغيلية (صيانة، نظافة، وغيرها).
 - خدمة الشكاوى والاقتراحات (Feedback Service): تمكين الطالب من تقديم شكوى أو اقتراح مع إمكانية إرفاق صور، ومتابعتها من قبل الإدارة.
 - خدمة الإشعارات والإعلانات (Notifications & Announcements Service): إدارة الإشعارات الموجهة للطلاب والإعلانات العامة للمدينة الجامعية.

الفصل الثاني:
الأدوات ولغات البرمجة المستخدمة

الفصل الثاني: الأدوات ولغات البرمجة المستخدمة

يوضح هذا الفصل التقنيات الأساسية المستخدمة في بناء وتطوير منصة سكني، بالإضافة إلى توضيح المواضيع التي استُخدمت فيها داخل المشروع، مع نظرة سريعة أولاً على مبررات اختيار كل تقنية بالنسبة للخدمة التي تخدمها.

التقنيات المستخدمة في المشروع ومبررات اختيارها

مبرر الاختيار	أين تُستخدم	التقنية
المرونة في تمثيل أكثر من نوع مستخدم (طالب/مدير) بحقول مختلفة دون تعقيد جداول علائقية، وسهولة العمل مع JSON بما يتوافق مع طبيعة REST API.	خدمة المصادقة (Auth Service)	Node.js / Express.js / MongoDB
بيانات علائقية واضحة البنية (غرف، طلبات، شكاوى، إشعارات) تستفيد من قيود التكامل المرجعي وسهولة الاستعلام عبر Entity Framework Core و LINQ.	Housing، Service Requests، Feedback، Notifications & Announcements	ASP.NET Core / SQL Server / EF Core
توجيه الطلبات إلى الخدمة الداخلية المناسبة وفق إعدادات JSON، مع دعم موازنة التحميل والأمان عبر JWT/OAuth.	بوابة موحدة أمام جميع الخدمات	Ocelot API Gateway
بناء واجهة تفاعلية قائمة على مكونات قابلة لإعادة الاستخدام، مع دعم الأنواع (Types) الذي يقلل الأخطاء البرمجية، وإدارة حالة وتنقل وتفاعل مع الخادم عبر مكتبات متخصصة خفيفة الوزن.	لوحة تحكم الإدارة (Web)	React / TypeScript (React Router, React Query, Zustand)
بناء تطبيقات متعددة المنصات من قاعدة كود واحدة بأداء قريب من التطبيقات الأصلية (Native).	تطبيق الطالب (Mobile)	Flutter / Dart
حزم كل خدمة مع اعتمادياتها ضمن حاوية معزولة موحدة التشغيل عبر البيئات المختلفة.	جميع الخدمات	Docker
إصدار رمز دخول قصير الأمد (Access Token) مع رمز تجديد (Refresh Token) لتفادي تكرار تسجيل الدخول دون المساس بالأمان.	خدمة المصادقة	JWT + Refresh Tokens
تجزئة كلمة المرور قبل التخزين، بحيث لا تُحفظ أي كلمة مرور كنص صريح ضمن قاعدة البيانات.	خدمة المصادقة	bcrypt

الجدول (1-2)

لغات وأطر العمل المستخدمة في تطوير قسم Back-end

1.2 لغة C#:



تم اعتماد هذه اللغة بشكل أساسي في بناء و تطوير هذه المنصة، حيث قمنا بالاعتماد على اصدار 12 في بناء الشيفرة البرمجية الخاصة بالمشروع. تتميز هذه اللغة بكونها من اكثر اللغات دعماً و انتشاراً في عالم البرمجة حيث تم طرحها من قبل شركة مايكروسوفت عام 2000 كلفة غرضية التوجه متعددة الاستعمالات ، وما لبثت أن اصبحت لغة برمجية لكافة المنصات وأنظمة التشغيل basic script و syntax script و اطرات العمل فهي تدعم التنسيق المختلفة مما يجعلها قابلة للترجمة و سهولة الاستخدام من قبل كافة المبرمجين القادمين من لغات مختلفة

تعتبر C# cross platform ، كما انها تحتوي على العديد من الادوات التي تساعد في بناء تطبيقات الويب مثل : sql server و توافقها مع Asp.net core، و لغة الاستعلام LINQ، و Entity frame . core



2.2 Net core :

تعتبر إحدى أحدث اطر العمل من مايكروسفت ، مفتوحة المصدر و مجانية بشكل كامل كما تمتاز بدعمها لعدد من أنظمة التشغيل مثل ويندوز و لينوكس ، فهي متعددة المنصات . يحتوي اطار العمل هذا على العديد من المكتبات الجديدة و المصممة بكفاءة عالية ضمن اي نظام تشغيل يدعمه إطار العمل هذا .

3.2 ASP .NET CORE :



عبارة عن اطار عمل مفتوح المصدر و عالي الأداء و متعدد الأنظمة الأساسية لبناء تطبيقات حديثة و متصلة بالسحابة و متصلة إطار العمل هذا من إنشاء تطبيقات و خدمات الويب و تطبيقات انترنت الأشياء ، بالإضافة لاستخدام أدوات التطوير المفضلة لديك على أنظمة التشغيل ويندوز و لينوكس ، والنشر على السحابة .

وتشمل بعض المميزات سنذكر امثلة منها:

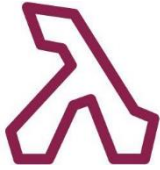
نظام التوجيه : حيث يتيح إمكانية تحويل الطلبات بناء على الرابط إلى المتحكم الملائم.

ربط النماذج : يقوم إطار العمل بعملية تعبئة البيانات القادمة من الواجهات في صفوف و

كائنات برمجية ليتم استخدامها في الشيفرة البرمجية من قبل المطورين.

حقن الاعتماديات : يقوم على مبدأ عزل مكونات التطبيق ضمن صفوف يغلفها واجهات برمجية من هذا المكون object بحيث يعتمد كل مكون على خاصية بمكون اخر وتقوم مكتبة الحقن بإنشاء و إعطائه لمكون الذي يعتمد عليه.

4.2 Language Integrated Query (LINQ) :



هي مكتبة تستخدم لتوحيد طريقة الاستعلام من مصادر مختلفة البيانات مثل Json و ذلك باستخدام كود سي شارب مباشرة و ذلك عن طريق كتابة تعابير Expressions و تحويلها للغة الاستعلام الأصلية وتنفيذها و من ثم إعادة النتيجة ككائنات ضمن لغة C# ، قمنا بالاعتماد إحدى تضمينات هذه المكتبة و الذي يدعى L-To-LINQ و ذلك من أجل كتابة الاستعلامات بلغة C#.

ومن ثم تقوم المكتبة بعملية تحسين لهذه الاستعلامات و من ثم إرسالها لقاعدة البيانات ليتم تنفيذها وتعبئة الجدول الناتج ضمن كائنات مناسبة ، حيث تعتمد بشكل رئيسي على ال foreach yield من بنيات Enumerable و Queryab .

5.2 Entity framework core (EF CORE) :

تم اعتماد مكتبة Entity framework core و ذلك لربط قاعدة بيانات SQL server مع تطبيق C# ، هذه المكتبة تعتمد مفهوم ORM والذي يعتمد على تحويل جداول قاعدة البيانات الى صفوف برمجية classes وبالعكس، كما نتيج لنا هذه المكتبة مزايا اخرى مثل إنشاء قاعدة البيانات انطلاقا من الصفوف البرمجية بالاضافة الى مراقبة التعديلات على هيكلية قاعدة البيانات schema والاستعلام من قاعدة البيانات باستخدام مكتبة LINQ.



6.2 SQL لغة الاستعلامات :



هي لغة برمجية تستخدم للتواصل مع البيانات المخزنة في نظام إدارة قواعد البيانات العلائقية . يشبه بناء جملة SQL اللغة الانكليزية، مما يجعل من السهل نسبيا الكتابة و القراءة و التفسير .

تستخدم العديد من أنظمة RDBMS SQL للوصول إلى البيانات الموجودة في الجداول. على سبيل المثال ، يعد SQLite نظاما لإدارة قواعد البيانات العلائقية، يحتوي SQLite

على الحد الأدنى من مجموعة أوامر SQL والتي هي نفسها في جميع أنظمة إدارة قواعد البيانات RDBMS. قد تستخدم أنظمة RDBMS الأخرى متغيرات أخرى.

7.2 متحكم الإصدارات GIT:

هو نظام تحكم في الإصدارات (Version Control System) موزع و مفتوح المصدر، يستخدم لتتبع التغيرات في ملفات الكود و المشاريع البرمجية خلال الوقت. الغرض الرئيسي هو تسجيل تاريخ التغيرات في الملفات (خاصة ملفات الكود) والسماح بالتعاون بين عدة مطورين دون تضارب. والميزة الأساسية ان يعمل بنظام موزع (Distributed) اي كل مطور لديه نسخة كاملة من تاريخ المشروع على جهاز.



لغات وأطر العمل المستخدمة في تطوير قسم Front-end

8.2 لغة (HTML (Hyper Text Markup Language):

هي لغة الترميز تستخدم مع لغة JavaScript, css للحصول على مواقع و صفحات ويب تفاعلية و تنفيذ الأوامر على تلك المواقع. تقوم بوصف بنية الصفحات هيكلياً تعرف متصفح الانترنت بالصور والفيديو و الكلمات التي يتم حيث اضافتها على الموقع, فيتم كتابة الصفحات بوسوم معينة فترسل HTML عام 2012 التي تحتوي على العديد من المميزات و الخصائص المفيدة.



بعض رموز لغة HTML:

<!Doctype>: اعلام المتصفح بإصدار لغة HTML المعتمد في الصفحة.

<body>: من الأقسام الرئيسية في الصفحة نضع بداخله محتوى الصفحة الذي سيظهر للمستخدم.

<head>: من الأقسام الرئيسية في الصفحة يستخدم اعلام المتصفح بكيفية عرض المحتوى و تضمين ملفات التصميم, سكريبتات الصفحة.....

<a>: لعرض رابط في الصفحة.

<div>: لتقسيم محتوى الصفحة.

<footer>: يوضع في اخر الصفحة و يوضع فيه اسم المؤلف, معلومات التواصل

9.2 CSS3 (Cascading Style Sheet):

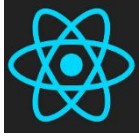
هي لغة توصيفيه لتنسيق صفحات الويب تهتم بشكل و تصميم الموقع. قبل البدء ببناء و تطوير تلك اللغة في عام 1996 من قبل اتحاد شبكة الويب العالمية C3W كانت صفحات الويب محدودة و بسيطة من حيث الشكل الوظيفة سمحت CSS بتصميم و تشكيل صفحات الويب بطرق متعددة لتتمكن من استيعاب مدى ابداع الشخص مثل القدرة على تحديد الهوامش و الخطوط و المسافات لعناصر الموقع.





10.2 لغة TypeScript:

هي لغة برمجية مفتوحة المصدر من تطوير شركة Microsoft، تعد امتداداً وتوسعة لـ JavaScript، حيث أضافت العديد من المزايا إليها، خاصة دعم الأنواع types الذي ساعد على تجنب الأخطاء البرمجية وتوفير شيفرة برمجية نقية قابلة للقراءة أكثر من شيفرة JavaScript العادية كما أنها أتاحت استخدام الواجهات interface



11.2 إطار العمل React:

هو إطار عمل JavaScript ومكتبة مفتوحة المصدر تم تطويرها بواسطة Facebook يتم استخدامه لبناء واجهات مستخدم تفاعلية وتطبيقات ويب بسرعة وكفاءة مع رمز أقل بكثير مما تفعله مع Vanilla javascript في React، تقوم بتطوير تطبيقاتك عن طريق إنشاء مكونات قابلة لإعادة الاستخدام يمكنك التفكير فيها على أنها كتل مستقلة. هذه المكونات عبارة عن أجزاء فردية من الواجهة النهائية، والتي، عند تجميعها، تشكل واجهة مستخدم التطبيق بالكامل.

يتمثل الدور الأساسي لـ React في تطبيق ما في التعامل مع طبقة العرض الخاصة بهذا التطبيق تماماً مثل view في نموذج تحكم عرض النموذج MVC من خلال توفير تنفيذ العرض الأفضل والأكثر كفاءة. بدلاً من التعامل مع واجهة المستخدم بأكملها كوحدة واحدة، يشجع React.js المطورين على فصل واجهات المستخدم المعقدة هذه إلى مكونات فردية قابلة لإعادة الاستخدام تشكل اللبنات الأساسية لواجهة المستخدم بأكملها. عند القيام بذلك، يجمع إطار عمل ReactJS بين سرعة وكفاءة JavaScript وطريقة فاعلة لمعالجة DOM عرض صفحات الويب بشكل أسرع وإنشاء تطبيقات ويب ديناميكية وسريعة الاستجابة.

بعض ميزات React:

- * المرونة: بالمقارنة مع أطر العمل والمكتبات الأخرى سوف نجد أن أكواد الـ React أسهل في الإصلاح وحتى في الكتابة، مما يوفر عليك الوقت ويزيد من مرونة الـ React.
 - * الأداء: تم تصميم الـ ReactJS لتوفير كفاءة عالية في الأداء، صمم المكتبة يوفر الـ Virtual DOM ودعم لجانب الخادم مما يجعل من المواقع والتطبيقات المعقدة تعمل بسرعة فائقة.
 - * لديهم مجتمع قوي وضخم: مثل إطار العمل Angular لدى React.
- ماهي المكونات Components:
- المكونات أو كما يطلق عليها اسم مكونات واجهة المستخدم، هو مصطلح يطلق على عنصر تم بناؤه بواسطة رايكت ويتم إعادة استخدامه في العديد من الصفحات. تتميز المكونات بالبساطة وسهولة التحكم وسوف توفر عليك الكثير من الوقت والجهد أثناء البرمجة ويمكنك أيضاً الاعتماد عليها في إنشاء تطبيقات بشكل كامل.

12.2 صيغة JSX:



أي موقع على شبكة الإنترنت يتكون من ملفات HTML، يقرأ المتصفح هذه الملفات ويعرضها لك على شاشة الجهاز الذي تستخدمه. يفضل استخدام طريقة الـ JSX في الكتابة لسهولة استخدامها ولأنها تساعد مكتبة رايكت على إظهار الأخطاء المفيدة والرسائل التحذيرية بشكل أفضل.

13.2 إطار العمل React-query:



React Query

مكتبة شهيرة لإدارة الحالات (state management) في تطبيقات React، تركز بشكل أساسي على إدارة البيانات غير المتزامنة (async data) والتفاعل مع الخوادم (server state)

ويبسّط عملية إدارة البيانات غير المتزامنة في تطبيقات React، ويوفر أدوات قوية للتفاعل مع APIs والخوادم، مع تحسين الأداء وتجربة المستخدم من خلال التخزين المؤقت الذكي وآليات التحديث التلقائي.

الميزات الرئيسية:

1. إدارة ذكية للبيانات: التخزين المؤقت caching، وإعادة الجلب refetching، والتحديث التلقائي.
2. مزامنة البيانات: مزامنة البيانات بين الخادم والعرض UI.
3. تحسين الأداء: تقليل الطلبات الزائدة عن الحاجة.
4. واجهة بسيطة: استخدام hooks مثل useQuery و useMutation.
5. إدارة الأخطاء: معالجة الأخطاء بطريقة مركزية.

14.2 مكتبة React-router:



React Router

React Router v6.4 هو إصدار رئيسي من مكتبة React Router، وهي أداة شائعة لإدارة التنقل Routing في تطبيقات React يُقدم هذا الإصدار العديد من الميزات الجديدة والتحسينات مقارنة.

بالإصدارات السابقة، مع التركيز على تحسين تجربة المطور وتوفير أدوات أكثر قوة لتحميل البيانات وإدارة الحالة. حيث يُقلل الحاجة إلى مكتبات خارجية لتحميل البيانات ومعالجة النماذج، ويُقدم تحسينات كبيرة للأداء وتجربة المطور.

15.2 مكتبة Zustand:



Zustand هي مكتبة خفيفة وبسيطة لإدارة حالة التطبيق State في تطبيقات JavaScript، وخاصة مع React. تتميز بالبساطة وسهولة الاستخدام مقارنة بمكتبات أخرى مثل Redux، ولا تحتاج إلى إعدادات أو ملفات كثيرة. تعتمد Zustand على إنشاء store يمكن لأي مكون في التطبيق الوصول إليه مباشرة، بدون الحاجة إلى Provider.

مميزات Zustand:

البساطة Zustand لا تحتاج إلى Dispatch, Actions, Reducers كل شيء يكون داخل store واحد، مما يقلل حجم الكود ويجعله أسهل في القراءة والصيانة. حل مشكلة Prop Drilling

في React العادي قد نحتاج لتمرير البيانات عبر عدة مكونات حتى تصل للمكون المطلوب Zustand تحل هذه المشكلة، حيث يمكن لأي مكون الوصول للحالة مباشرة من الـ store بدون تمرير props. أداء ممتاز: Zustand تعيد رسم المكونات فقط عند تغير الجزء المستخدم من الحالة، مما يحسن الأداء ويقلل إعادة الرسم غير الضرورية.

لا تحتاج Provider

بعكس Redux و Context API، لا تحتاج Zustand إلى لف التطبيق داخل Provider. يمكنك استخدام الـ store مباشرة داخل أي مكون.

مناسبة للتطبيقات الصغيرة والمتوسطة Zustand خيار ممتاز للتطبيقات الصغيرة و المتوسطة ولوحات التحكم Dashboards صفحات الإدارة Admin Panels كما يمكن استخدامها في تطبيقات كبيرة مع تنظيم جيد دعم الإضافات Middleware Zustand تدعم إضافات مثل: حفظ البيانات في LocalStorage أدوات التصحيح والتتبع DevTools

16.2 مكتبة TailwindCSS:



Tailwind CSS هو إطار عمل Utility-First CSS Framework يُستخدم لتصميم واجهات المستخدم بسرعة وكفاءة دون الحاجة لكتابة CSS تقليدي. يعتمد على فكرة الفئات الجاهزة" مثل (p-4, text-blue-500, flex) التي تُطبق مباشرة في HTML، مما يقلل الحاجة لكتابة ملفات.

17.2 مكتبة Material UI:



MUI المعروفة سابقاً باسم Material-UI هي مكتبة شهيرة لمكونات واجهة المستخدم UI Components مبنية على تصميم Material Design من Google تُستخدم مع إطارات عمل مثل React لإنشاء تطبيقات ويب وتطبيقات جوال بتصميم عصري وسهل الاستخدام.

18.2 صيغة JSON:



صيغة (JSON (JavaScript Object Notation هي تنسيق نصي خفيف الوزن يستخدم لتبادل البيانات بين الخادم والعميل أو بين أنظمة مختلفة. وهي سهلة القراءة والكتابة لكل من البشر والآلات. تتكون JSON من أزواج مفتاح/قيمة (مثل الكائنات في JavaScript وقوائم مرتبة (مثل المصفوفات).

19.2 إطار العمل Flutter:



إطار عمل framework مفتوح المصدر طورته Google لبناء تطبيقات متنوعة Cross-Platform بكود واحد يعمل على عدة منصات بنفس الجودة والأداء. يعتبر إطار عمل لتطوير التطبيقات باستخدام لغة Dart أيضاً من تطوير Google.

يهدف لإنشاء تطبيقات iOS، Android، الويب، سطح المكتب (Windows/macOS/Linux) من قاعدة كود واحدة. أداء قريب من التطبيقات الأصلية (Native) بسبب عدم اعتماده على متصفح أو جسر (Bridge) مثل بعض الأطر الأخرى.

20.2 مكتبة Ocelot:

هي بوابة API (Gateway) مفتوحة المصدر مبنية لمنصة NET Core، تعمل كطبقة وسيطة بين العملاء والخدمات المصغرة (Microservices) تقوم باستقبال الطلبات وتوجيهها إلى الخدمات الداخلية وفق إعدادات مرنة.



أبرز ميزاتها:

- التوجيه (Routing): إعادة توجيه المسارات بناءً على إعدادات JSON.
- اكتشاف الخدمات: تكامل مع أدوات مثل Consul لتحديد مواقع الخدمات ديناميكياً.
- موازنة التحميل: توزيع الطلبات بين نسخ الخدمة مثل (RoundRobin).
- الأمان: دعم JWT و OAuth للتحكم في الوصول.
- التحكم في التدفق: تحديد معدل الطلبات وتطبيق نمط "قاطع الدائرة" (Circuit Breaker) لتحمل الأعطال.

المحررات والتطبيقات المستخدمة في عمليات التطوير

المحرر VS CODE (visual studio code):



Visual Studio Code يُختصر (VS Code) هو محرر أكواد خفيف وسريع من تطوير مايكروسوفت، يدعم عشرات لغات البرمجة مثل JavaScript، Python، C++، HTML/CSS وغيرها يُعتبر من أكثر أدوات المطورين شعبية بسبب سهولة استخدامه، التوثيق الغني، والمجتمع الضخم حوله.

المحرر (SQL Server Management Studio) SMSS:



SQL Server Management Studio (SSMS) هو أداة متكاملة تُقدمها مايكروسوفت لإدارة وتطوير قواعد البيانات على منصة Microsoft SQL Server و Azure SQL Database.

هو واجهة رسومية (GUI) مصممة لتبسيط إدارة قواعد بيانات SQL Server حيث يستخدمه مطورو قواعد البيانات (DB Developers)، مسؤولو قواعد البيانات (DBAs)، محللو البيانات.

برنامج POST MAN:



POSTMAN

Postman هو أداة شائعة تُستخدم لاختبار وتطوير واجهات برمجة التطبيقات (APIs) يُستخدم من قبل المطورين لإنشاء وإرسال طلبات HTTP إلى الخوادم (Servers) وفحص الاستجابات (Responses) بسهولة.

برنامج GitHub:



منصة استضافة شائعة تُستخدم لإدارة المشاريع البرمجية والتعاون بين المطورين باستخدام نظام التحكم في الإصدار Git يُعتبر منصة سحابية تستضيف مستودعات (Repositories) تخزين كود المصدر. وتُسهل التعاون بين المطورين عبر أدوات مثل:

تتبع المشاكل (Issues) وطلبات السحب (Pull Requests) لمراجعة التغييرات وأدوات CI/CD مثل (GitHub Actions).

برنامج Trello:



Trello هو أداة لإدارة المشاريع والمهام بطريقة مرئية وسهلة الاستخدام، تعتمد على مفهوم اللوحات (Boards) والبطاقات (Cards) لتنسيق العمل والتعاون بين الفرق.

المحرر Android studio:



Android Studio هو بيئة التطوير الرسمية المقدمة من Google لبناء تطبيقات Android، وهو مبني على أساس IntelliJ IDEA من JetBrains يُعتبر محرر ذكي واقتراحات أكواد (Code Completion)، وتصحيح الأخطاء، ودعم للغات متعددة. يدعم محاكي أندرويد (Emulator) وتشغيل التطبيق على أجهزة افتراضية بمواصفات قابلة للتخصيص. يحتوي على أدوات تحليل الأداء ومراقبة استهلاك الذاكرة، وحدة المعالجة المركزية CPU، والبطارية.

الفصل الثالث:
تحليل المشروع وتوثيق البيانات

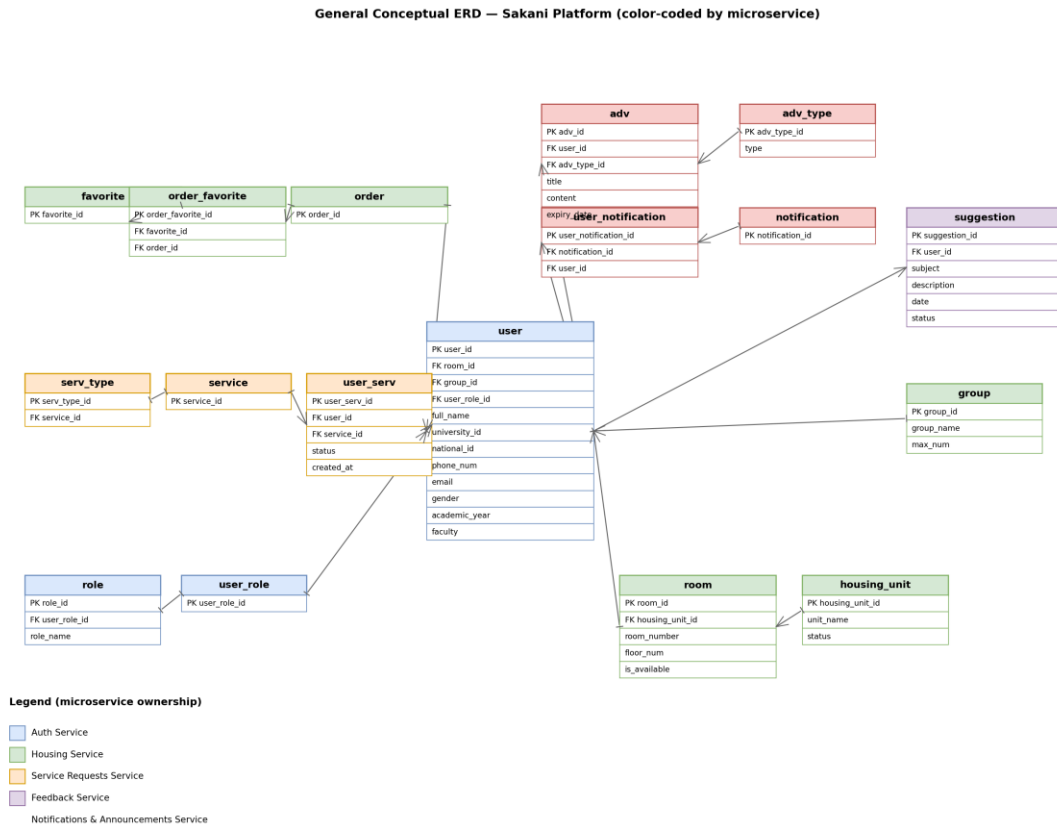
الفصل الثالث: تحليل المشروع وتوثيق البيانات

ملاحظة منهجية حول استخدام ERD

على الرغم من أن معمارية الخدمات المصغرة لا تعتمد من حيث المبدأ على قاعدة بيانات مركزية واحدة، فقد استخدم مخطط الكيانات والعلاقات (ERD) في هذا المشروع كأداة مساعدة لتبسيط فهم النظام وتوضيح العلاقات بين الكيانات المختلفة على المستوى العام (Conceptual Model)، وليس كتمثيل لقاعدة بيانات فعلية موحدة. أما في التطبيق الفعلي، فتمتلك كل خدمة مخطط ERD خاصاً بها وقاعدة بيانات مستقلة، كما هو موضح لاحقاً في هذا الفصل.

مخطط ERD العام (المفاهيمي)

يوضح الشكل التالي المخطط المفاهيمي العام لكيانات النظام، مع تلوين كل مجموعة كيانات بحسب الخدمة المصغرة المسؤولة عنها ضمن التصميم النهائي المعتمد للخدمات الخمس:



الشكل (1-3)

المخطط المفاهيمي العام (ERD) لمنصة سكني، مع تلوين الكيانات حسب الخدمة المصغرة المسؤولة

مخططات كل خدمة على حدة

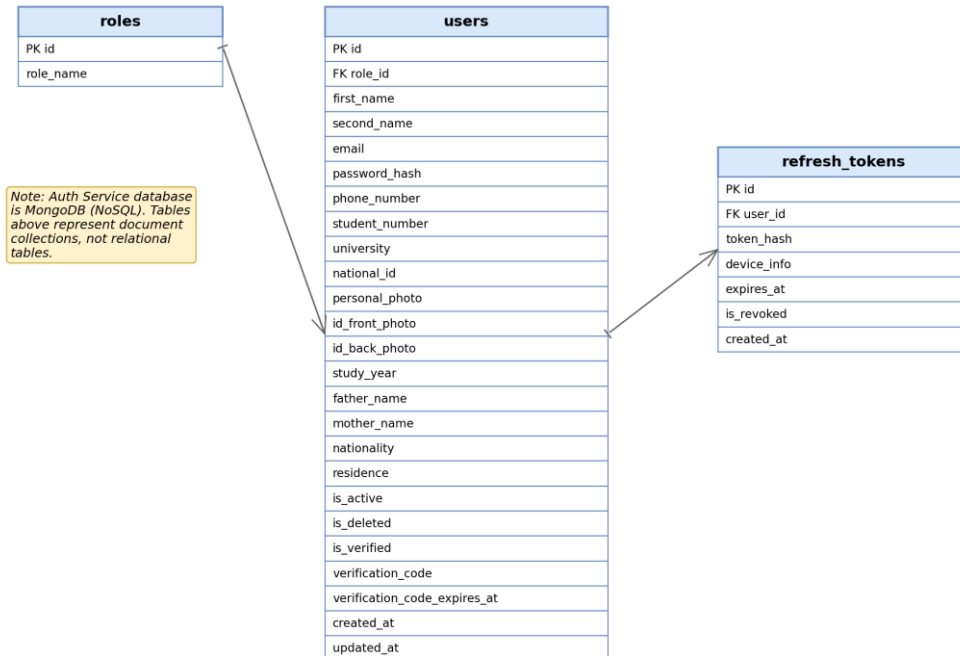
فيما يلي مخطط الكيانات والعلاقات (ERD) الفعلي، ومخطط حالات الاستخدام (Use Case Diagram)، لكل خدمة من الخدمات الخمس المكوّنة لمنصة سكني.

أ. خدمة المصادقة (Auth Service)

تتولى هذه الخدمة إدارة هوية المستخدمين بشكل مستقل عن باقي الخدمات: التسجيل، تسجيل الدخول، التحقق من الهوية، إدارة الأدوار، وإصدار وتجديد رموز الدخول. وفيما يلي منطق العمل الأساسي الذي تتبعه:

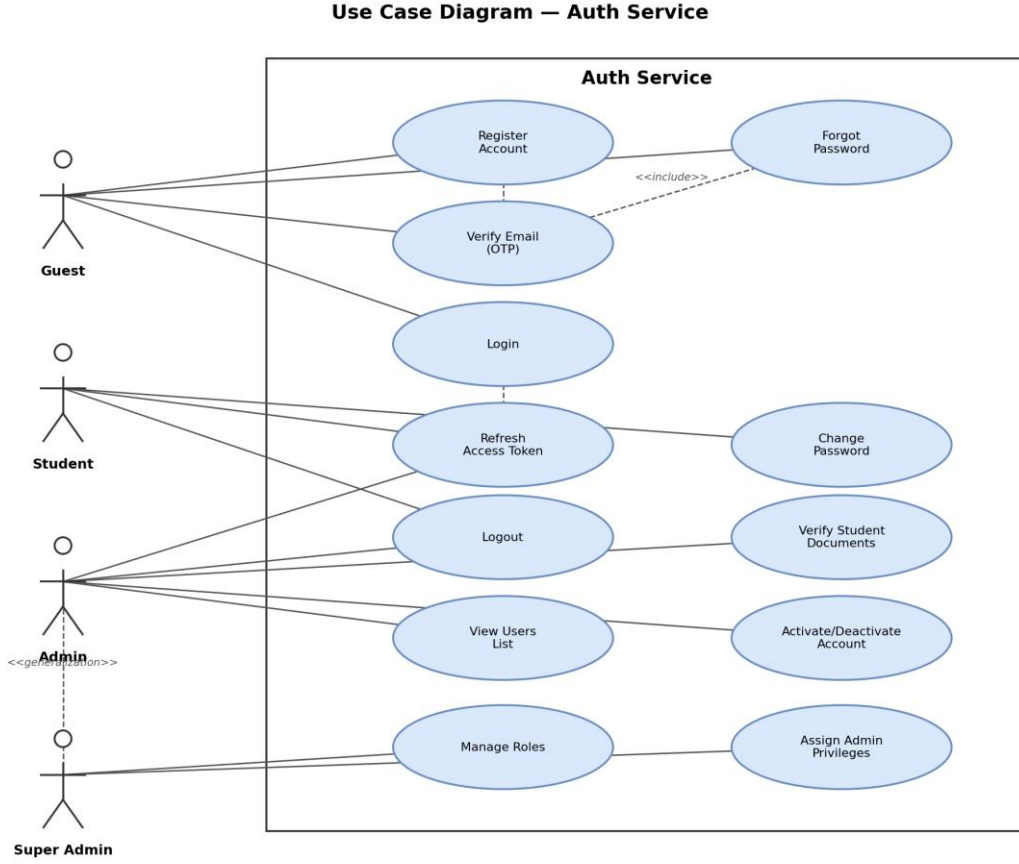
- يبدأ الطالب بالتسجيل عبر إدخال بياناته الأساسية على مرحلتين، ثم تُرسل الخدمة رمز تحقق (OTP) إلى بريده الجامعي لتأكيد الحساب قبل تفعيله.
- عند تسجيل الدخول، تتحقق الخدمة من كلمة المرور بعد تجزئتها (bcrypt) دون تخزين أي نص صريح، وتصدر Access Token قصير الأمد مع Refresh Token يتيح تجديد الجلسة دون تكرار تسجيل الدخول.
- كل مستخدم مرتبط بدور واحد فقط ضمن هذا التصميم (طالب/مدير/مدير عام)، وهو قرار مقصود من فريق المشروع لتبسيط نظام الصلاحيات بدل اعتماد علاقة Many-to-Many كاملة.
- تراجع الإدارة الوثائق الثبوتية التي يرفعها الطالب (صورة شخصية ووثائق الهوية) وتتحقق منها يدوياً قبل تفعيل الحساب بشكل نهائي.

ERD — Auth Service



الشكل (3-2)

مخطط ERD لخدمة المصادقة (Auth Service) — قاعدة بيانات MongoDB



الشكل (3-3)

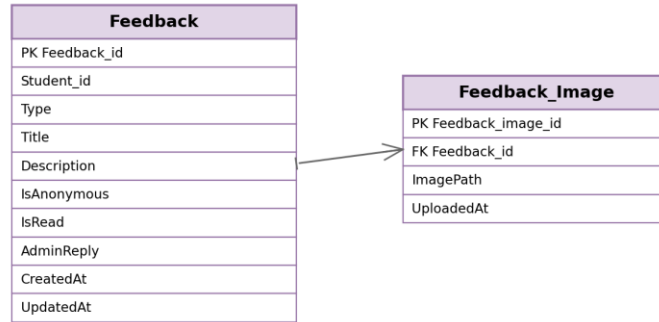
مخطط حالات الاستخدام (Use Case) لخدمة المصادقة

ب. خدمة الشكاوى والاقتراحات (Feedback Service)

تمكن هذه الخدمة الطالب من تقديم شكاوى ومقترحات مباشرة إلى الإدارة، ومتابعتها حتى الرد عليها. منطق العمل الأساسي فيها:

- يملأ الطالب نموذج شكاوى أو اقتراح (عنوان، وصف، نوع)، مع إمكانية إرفاق صور توضيحية، والاختيار بين إرسالها باسمه أو بشكل مجهول (IsAnonymous).
- يُخزن معرف الطالب (Student_id) كحقل عادي غير مرتبط بمفتاح أجنبي مباشر لقاعدة بيانات خدمة أخرى، ويتحقق من صحته عبر نداء API لخدمة المصادقة وقت الإرسال، حفاظاً على استقلالية قواعد البيانات بين الخدمات.
- تستعرض الإدارة جميع الشكاوى الواردة، وتستطيع تصفيتها حسب النوع أو الحالة، والرد المباشر على كل شكاوى (AdminReply) وتحديد كمقروءة (IsRead).

ERD — Feedback Service

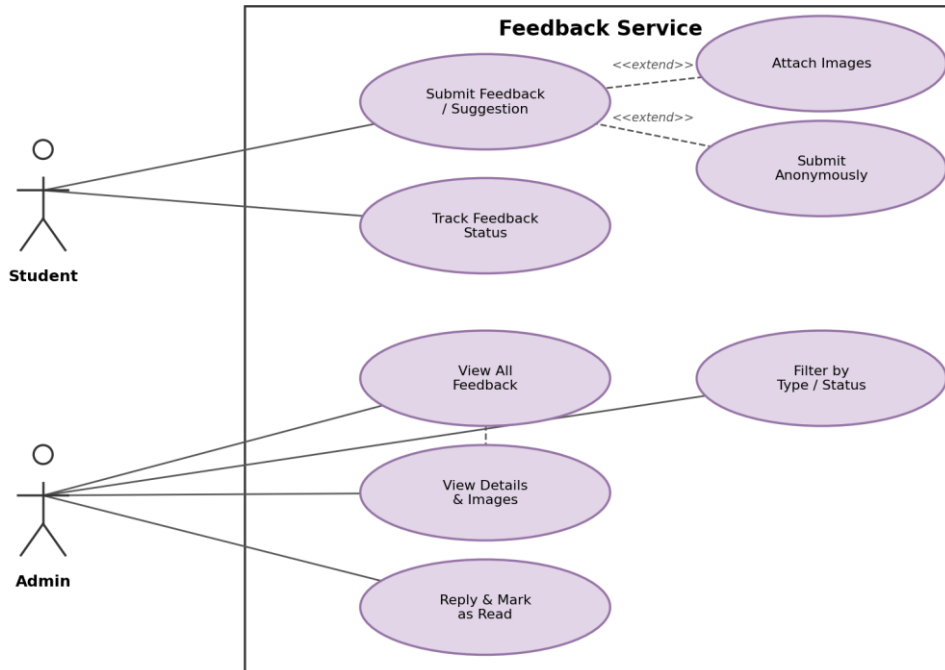


Note: Student_id is a plain field (not a direct FK across service databases), verified via a call to the Auth Service at creation time.

الشكل (3-4)

مخطط ERD لخدمة الشكاوى والاقتراحات

Use Case Diagram — Feedback Service



الشكل (3-5)

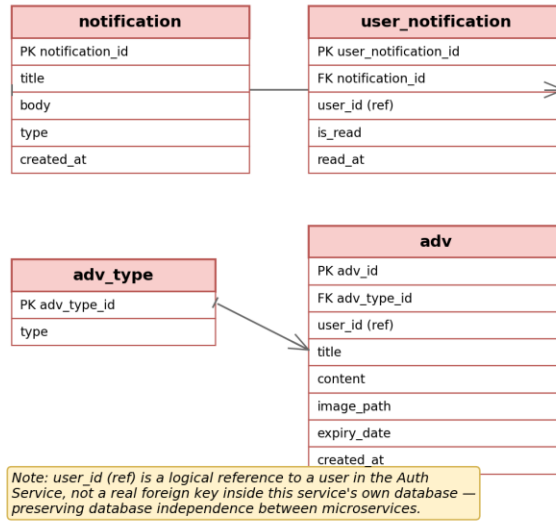
مخطط حالات الاستخدام لخدمة الشكاوى والاقتراحات

ج. خدمة الإشعارات والإعلانات

تدير هذه الخدمة قناتي تواصل من الإدارة نحو الطلاب: الإشعارات الفردية والإعلانات العامة. منطق العمل الأساسي فيها:

- تُصنّف الإعلانات ضمن أنواع محددة مسبقًا (adv_type)، وتنشئها الإدارة مع تحديد تاريخ انتهاء صلاحية (expiry_date) بحيث تختفي تلقائيًا من واجهة الطالب بعد ذلك التاريخ.
- تستطيع الإدارة إرسال إشعار مباشر إلى طالب واحد، أو مجموعة محددة من الطلاب، أو بثّه جماعيًا (Broadcast) لكل الطلاب دفعة واحدة.
- يُخزّن الحقل user_id في جدولي user_notification وadv كمرجع منطقي فقط لمستخدم في خدمة المصادقة، وليس كمفتاح أجنبي فعلي داخل قاعدة بيانات هذه الخدمة، اتساقًا مع مبدأ استقلالية الخدمات.
- يتابع الطالب إشعاراته ضمن قائمة مرتّبة زمنيًا، ويمكنه تحديد أي إشعار كمقروء (is_read) بعد الاطلاع عليه.

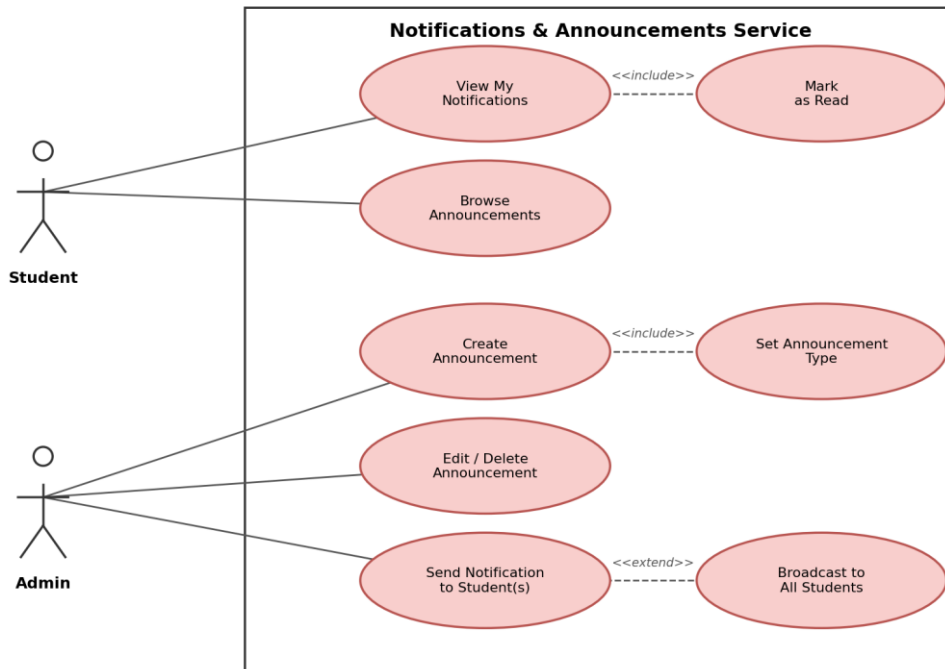
ERD — Notifications & Announcements Service



الشكل (3-6)

مخطط ERD لخدمة الإشعارات والإعلانات

Use Case Diagram — Notifications & Announcements Service



الشكل (3-7)

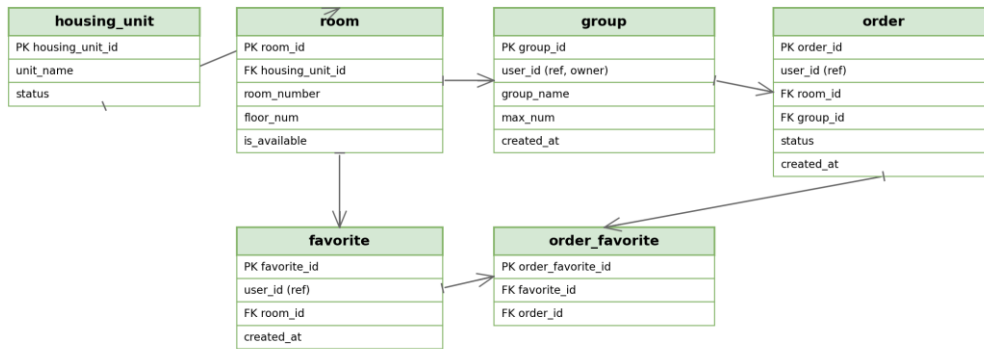
مخطط حالات الاستخدام لخدمة الإشعارات والإعلانات

د. خدمة السكن

تدير هذه الخدمة دورة حياة السكن الجامعي للطلاب بالكامل، من تصفح الغرف حتى تخصيصها فعليًا. منطق العمل الأساسي فيها:

- يتصفح الطالب الغرف المتاحة، ويمكنه إضافة أي غرفة إلى المفضلة (favorite) قبل اتخاذ قرار الحجز النهائي.
- ينشئ الطالب مجموعة سكن (رفقاء سكن) ويدعو أعضاء آخرين للانضمام إليها؛ وتُعطّل الدعوة تلقائيًا عند وصول عدد الأعضاء إلى الحد الأقصى المسموح به (max_num).
- يقدم الطالب طلب حجز (order) مرتبطًا بغرفة و/أو مجموعة سكن، وتنتقل حالة الطلب عبر مراحل محددة (قيد المراجعة، مقبول، مرفوض).
- تراجع الإدارة الطلبات الواردة وتخصّص الغرفة فعليًا للمجموعة المقبولة، مع تحديث خريطة إشغال المباني والغرف تلقائيًا بناءً على القرار.

ERD — Housing Service

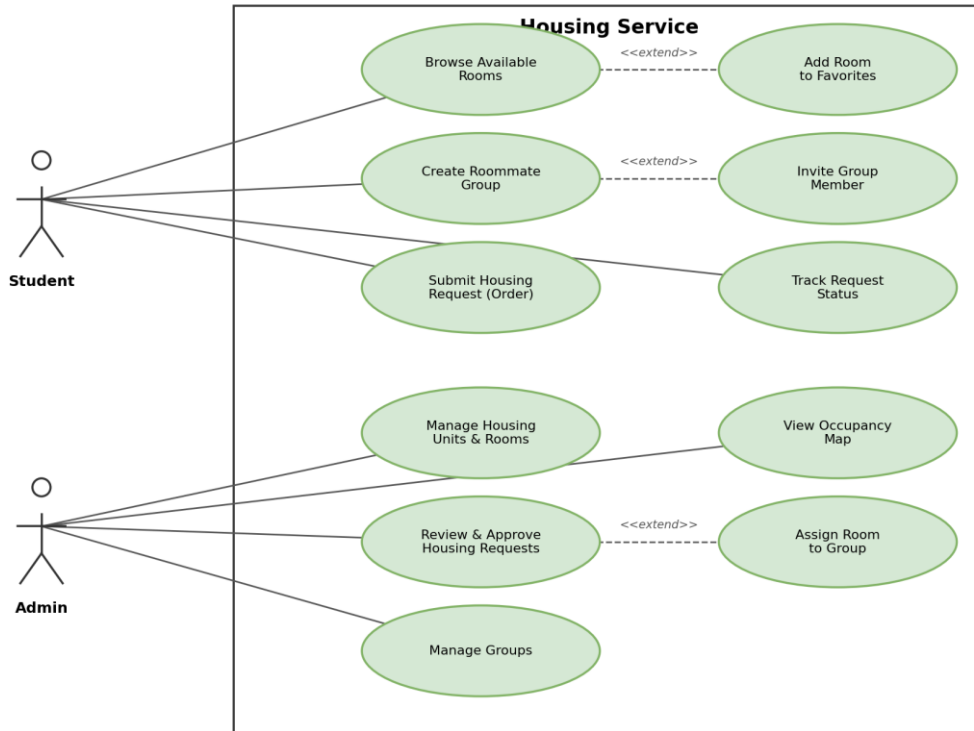


Note: user_id (ref) is a logical reference to a user in the Auth Service, not a real foreign key inside this service's own database. A booking order (order) is linked to a room and/or a roommate group, and a student may favorite a room before booking it (favorite -> order_favorite -> order).

الشكل (3-8)

مخطط ERD لخدمة السكن

Use Case Diagram — Housing Service



الشكل (3-9)

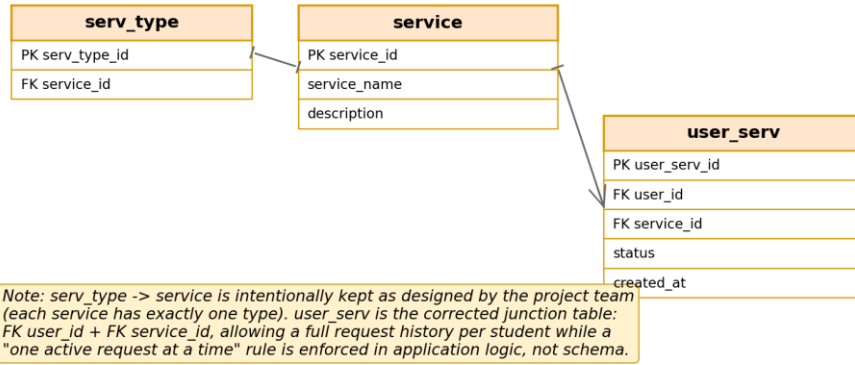
مخطط حالات الاستخدام لخدمة السكن

هـ. خدمة طلبات الخدمة

تستقبل هذه الخدمة طلبات الطلاب المتعلقة بالخدمات التشغيلية داخل المدينة الجامعية (كالصيانة والنظافة) وتتابعها حتى الإنجاز. منطق العمل الأساسي فيها:

- يختار الطالب نوع الخدمة المطلوبة من كتالوج محدد مسبقاً من قبل الإدارة (علاقة $service \rightarrow serv_type$ بنمط واحد لواحد، معتمدة عمداً بهذا الاتجاه ضمن تصميم فريق المشروع).
- عند إرسال الطلب، تتحقق الخدمة أولاً مما إذا كان لدى الطالب طلب «نشط» حالياً (Pending أو In Progress)؛ فإذا كان الأمر كذلك تُرفض العملية برسالة توضيحية، وذلك لمنع تكرار الطلبات (Spam) من الطالب نفسه — علماً أن هذه القاعدة تُطبّق على مستوى منطق التطبيق لا على مستوى بنية الجدول.
- تُعرض الطلبات على الإدارة ضمن لوحة Kanban منظمّة حسب الحالة، ما يتيح تتبّع دورة حياة كل طلب من الاستلام حتى الإنجاز بصرياً وبسهولة.
- يحتفظ النظام بالسجل الكامل لطلبات كل طالب عبر الزمن، بفضل تصحيح جدول $user_serv$ ليصبح جدول ربط قياسي (FK $user_id$ + FK $service_id$) بدل الحقل المفرد في التصميم الأولي.

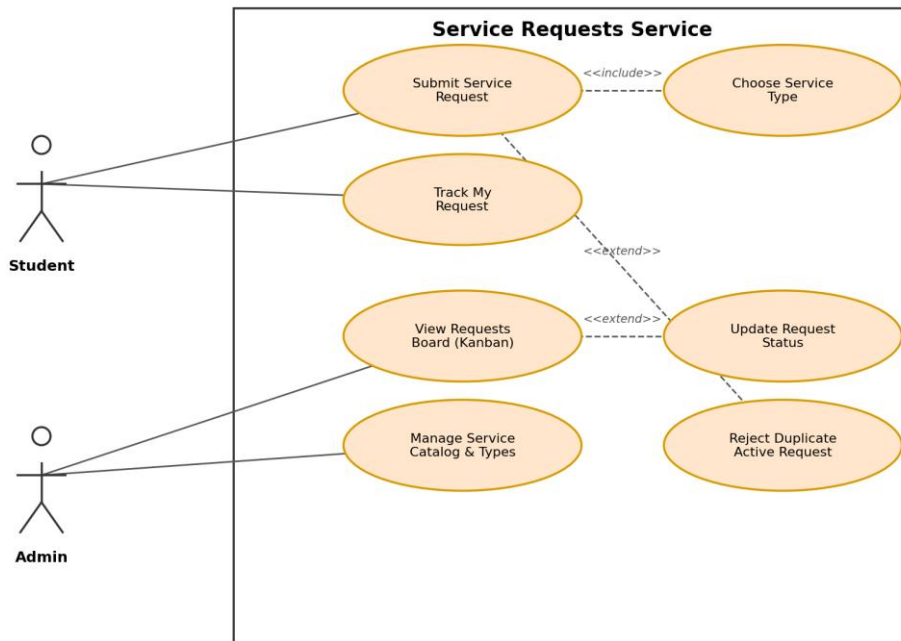
ERD — Service Requests Service



الشكل (3-10)

مخطط ERD لخدمة طلبات الخدمة — يوضّح تصحيح جدول user_serv ليصبح جدول ربط سليم

Use Case Diagram — Service Requests Service



الشكل (3-11)

مخطط حالات الاستخدام لخدمة طلبات الخدمة

جدول العلاقات بين الجداول

يلخّص الجدول التالي طبيعة العلاقة بين كل زوج من الكيانات ضمن المخطط المفاهيمي العام، مع توضيح مختصر لمعناها العملي:

الجدول	نوع العلاقة	شرح العلاقة
role — user_role	One-to-One	كل مستخدم مرتبط بدور واحد فقط ضمن هذا التصميم — قرار مقصود لتبسيط نظام الصلاحيات بدل اعتماد علاقة Many-to-Many كاملة.
user_role — user	One-to-Many	كل سجل أدوار يمكن أن يرتبط بعدة مستخدمين.
serv_type — service	One-to-One (تسلسلي)	علاقة معتمدة عمدًا بهذا الاتجاه ضمن تصميم فريق المشروع؛ كل نوع خدمة مرتبط بخدمة واحدة.
service — user_serv	One-to-Many	الخدمة الواحدة (مثل صيانة مكيف) يمكن أن تُطلب من عدة مستخدمين عبر عدة سجلات.
user — user_serv	One-to-Many	جدول وسيط مُصنَّح: يسمح للمستخدم بامتلاك أكثر من طلب خدمة عبر الزمن (سجل تاريخي)، مع تطبيق قاعدة «طلب نشط واحد بالوقت نفسه» على مستوى منطق التطبيق لا بنية الجدول.
— favorite order_favorite — order	One-to-Many Many-to- / One	جدول وسيط قياسي يربط تفضيلات الطالب بطلبات الحجز.
order — user	One-to-Many	كل مستخدم يمكن أن يقدّم عدة طلبات حجز عبر الزمن.
user — room	Many-to-One	عدة طلاب يمكن أن يشتركوا بنفس الغرفة.
room — housing_unit	Many-to-One	عدة غرف تنتمي لنفس الوحدة السكنية (المبنى).
user — group	Many-to-One	عدة طلاب ينتمون لنفس مجموعة السكن (رفقاء السكن)، بحدّ أقصى (max_num).
user — suggestion	One-to-Many	المستخدم الواحد يمكن أن يقدّم عدة شكاوى/اقتراحات.
— user user_notification notification —	One-to-Many Many-to- / One	جدول وسيط يسمح بإرسال نفس الإشعار لعدة مستخدمين.
user — adv	One-to-Many	المدير (كمستخدم) ينشر عدة إعلانات.
adv — adv_type	Many-to-One	عدة إعلانات يمكن أن تشترك بنفس النوع.
Feedback — Feedback_Image	One-to-Many	الشكوى الواحدة يمكن أن تحوي عدة صور مرفقة.

الجدول	نوع العلاقة	شرح العلاقة
housing_unit — room	One-to-Many	المبنى/الوحدة السكنية الواحدة تضم عدة غرف.
room — group	One-to-Many	الغرفة الواحدة يمكن أن تُخصَّص لمجموعة سكن واحدة تضم عدة طلاب (بحدٍّ أقصى max_num).
group — order	One-to-Many	مجموعة السكن الواحدة يمكن أن يكون لها عدة طلبات حجز (Orders) عبر الزمن.
room — favorite	One-to-Many	الغرفة الواحدة يمكن أن يُضيفها عدة طلاب إلى المفضلة قبل الحجز.
— favorite order_favorite — order	One-to-Many One-to- / Many	جدول وسيط يربط تفضيل الطالب بطلب الحجز الفعلي.
serv_type — service	One-to-One (تسلسلي)	علاقة معتمدة عمداً بهذا الاتجاه ضمن تصميم فريق المشروع؛ كل نوع خدمة مرتبط بخدمة واحدة (مكررة هنا ضمن سياق خدمة Service Requests المستقلة).
service — user_serv	One-to-Many	جدول وسيط مُصحَّح: الخدمة الواحدة يمكن أن تُطلب من عدة مستخدمين عبر عدة سجلات، مع تطبيق قاعدة «طلب نشط واحد بالوقت نفسه» على مستوى منطق التطبيق لا بنية الجدول.

الفصل الرابع:
عرض واجهات المشروع

الفصل الرابع: عرض واجهات المشروع

تم تصميم جميع واجهات المشروع كنماذج ثابتة (Static Mockups) باستخدام HTML و CSS، اعتمادًا على نظام التصميم الموحد الموضح في الفصل الأول، تمهيدًا لتنفيذها فعلي لاحقًا عبر Flutter للموبايل و React للوحة التحكم. يعرض هذا الفصل كل شاشة من شاشات المنصة مرقمة، مع شرح مختصر لوظيفتها.

أولاً: تطبيق الموبايل (Flutter) — واجهة الطالب

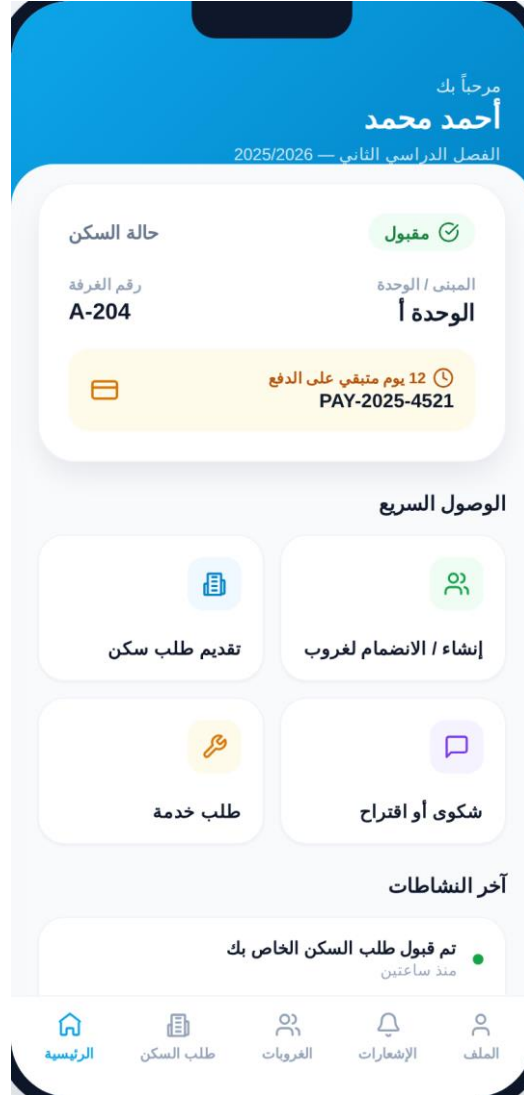
يضم تطبيق الموبايل 17 شاشة تغطي رحلة الطالب الكاملة، من التسجيل والدخول إلى إدارة السكن والخدمات والتواصل مع الإدارة:

1. تسجيل الدخول

الشكل (4-1): شاشة تسجيل الدخول

شاشة الدخول إلى التطبيق عبر البريد الجامعي وكلمة المرور، مع رابط لاستعادة كلمة المرور وآخر للانتقال إلى إنشاء حساب جديد.

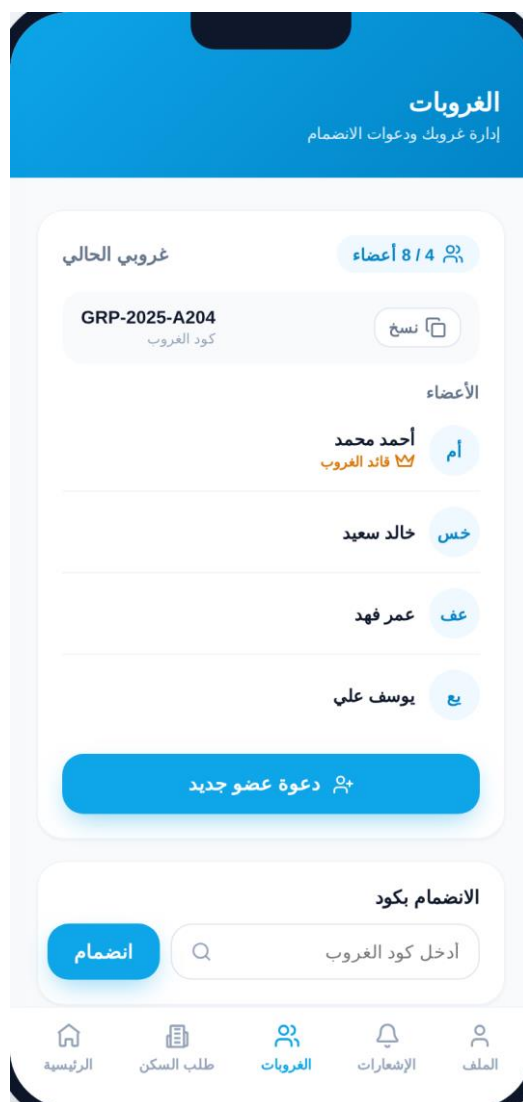
2. الرئيسية



الشكل (4-2): شاشة الرئيسية

الصفحة الرئيسية للطالب، وتعرض ملخصاً سريعاً لحالته السكنية وأهم الاختصارات نحو الخدمات المتاحة.

3. الغروبات



الشكل (3-4): شاشة الغروبات

شاشة إدارة مجموعة السكن (رفقاء السكن)، تتيح للطالب عرض أعضاء مجموعته الحالية ودعوة أعضاء جدد ضمن الحد الأقصى المسموح.

4. طلب السكن

الشكل (4-4): شاشة طلب السكن

شاشة تقديم طلب سكن جديد بأسلوب Stepper متعدد الخطوات، لتسهيل تعبئة بيانات الطلب تدريجيًا.

5. طلب خدمة



الشكل (4-5): شاشة طلب خدمة

شاشة استعراض الخدمات المتاحة (كالصيانة والنظافة) وتقديم طلب خدمة جديد من خلالها.

6. الشكاوى والاقتراحات



الشكل (4-6): شاشة الشكاوى والاقتراحات

شاشة الشكاوى والاقتراحات، تعرض للطلاب سجل ما قدّمه سابقاً وحالته الحالية.

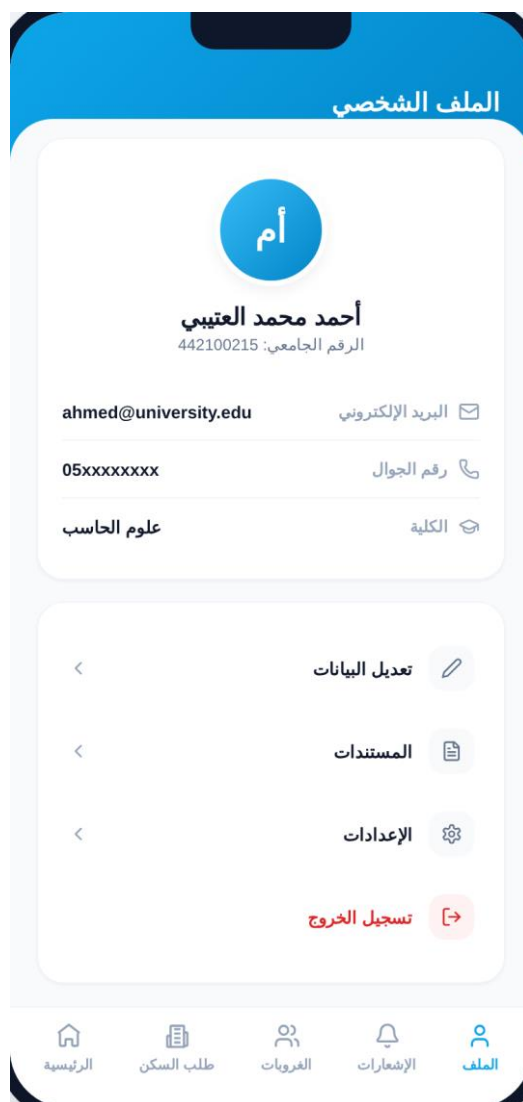
7. الإشعارات



الشكل (4-7): شاشة الإشعارات

شاشة الإشعارات الخاصة بالطالب، وتعرض الإشعارات الواردة من الإدارة مرتبة زمنياً.

8. الملف الشخصي



الشكل (8-4): شاشة الملف الشخصي

شاشة الملف الشخصي للطالب، وتعرض بياناته الأساسية وروابط الوصول السريع للإعدادات وتعديل البيانات.

9. إنشاء حساب (٢/١) البيانات الأساسية

الشكل (9-4): شاشة إنشاء حساب (٢/١) البيانات الأساسية

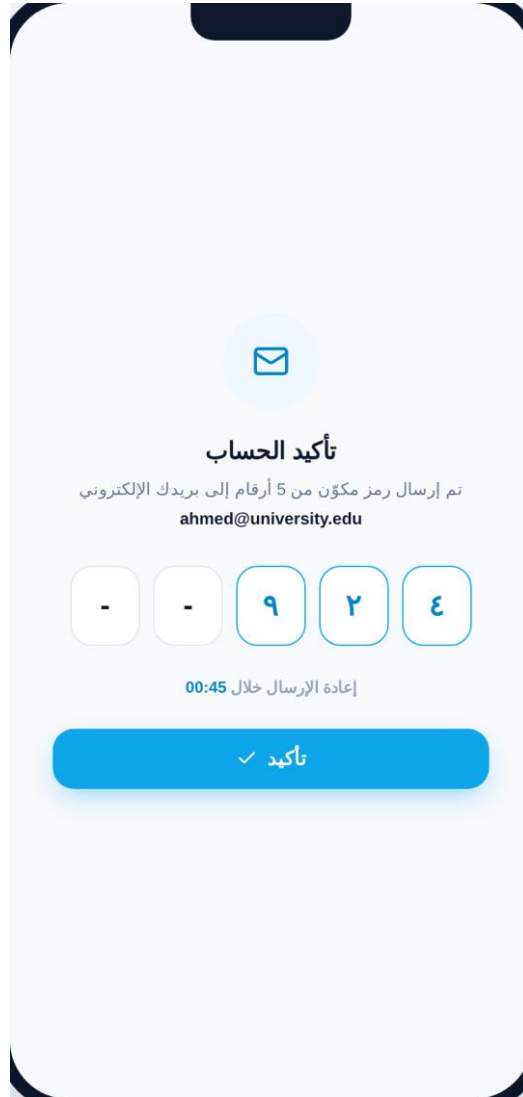
الخطوة الأولى من نموذج إنشاء حساب جديد، وتشمل إدخال البيانات الأساسية للطالب.

10. إكمال المعلومات (٢/٢)

الشكل (10-4): شاشة إكمال المعلومات (٢/٢)

الخطوة الثانية من إنشاء الحساب، لإكمال باقي البيانات المطلوبة (الجامعية والشخصية) قبل التفعيل.

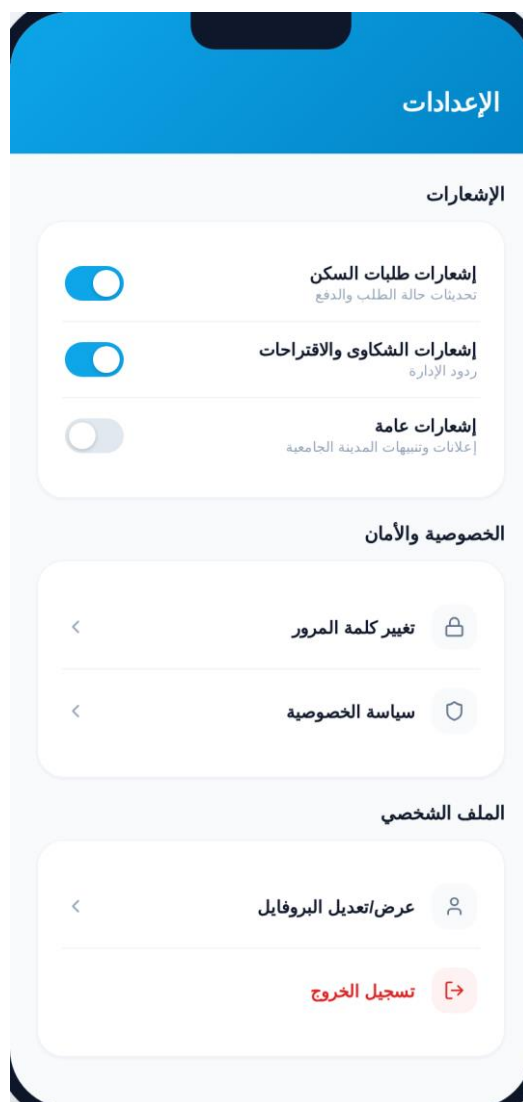
11. تأكيد الحساب (كود التحقق)



الشكل (4-11): شاشة تأكيد الحساب (كود التحقق)

شاشة التحقق من الحساب عبر رمز تأكيد (OTP) يُرسل إلى البريد الإلكتروني للطالب.

12. الإعدادات



الشكل (4-12): شاشة الإعدادات

شاشة إعدادات التطبيق العامة، وتشمل خيارات الحساب والتنبيهات وتسجيل الخروج.

13. طلب خدمة جديدة (تفاصيل الطلب)

طلب خدمة جديدة
اختر نوع الخدمة واكتب التفاصيل

نوع الخدمة

كهرباء سباكة إنترنت
مفاتيح تنظيف أخرى

وصف المشكلة
اشرح المشكلة بالتفصيل...

إرفاق صورة (اختياري)

إرسال الطلب

الشكل (13-4): شاشة طلب خدمة جديدة (تفاصيل الطلب)

شاشة تفاصيل طلب خدمة جديد، تتيح تحديد نوع الخدمة ووصف المشكلة بدقة أكبر.

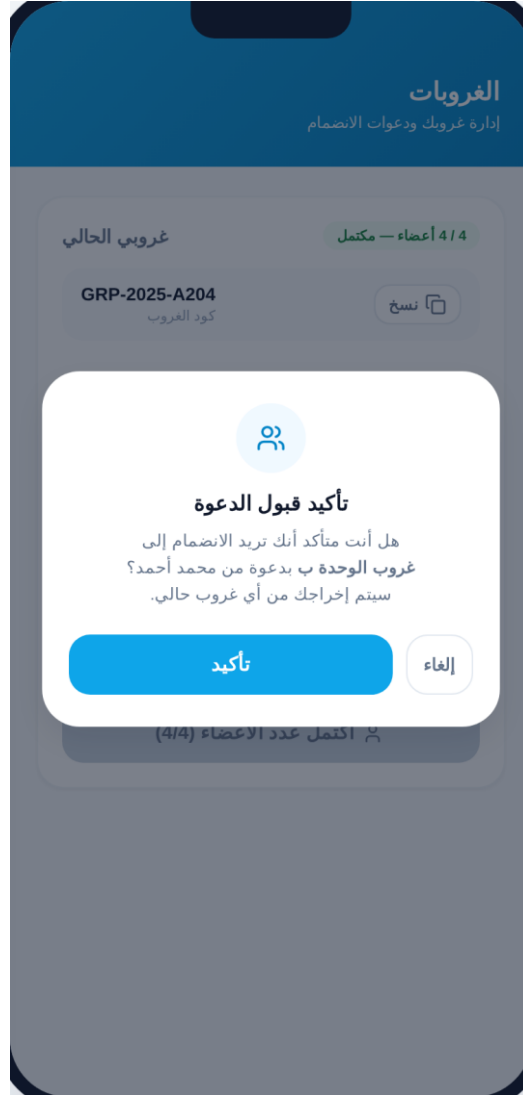
14. شكوى أو اقتراح جديد

The screenshot shows a mobile application interface for submitting a complaint or suggestion. At the top, there is a blue header with the title "شكوى أو اقتراح جديد" (New Complaint or Suggestion) and a subtitle "شاركنا رأيك أو مشكلتك" (Share your opinion or problem with us). Below the header, there is a section labeled "النوع" (Type) with two buttons: "اقتراح" (Suggestion) and "شكوى" (Complaint). The "شكوى" button is highlighted in blue. Below this, there is a section labeled "العنوان" (Title) with a text input field containing the placeholder "مثال: مشكلة في التكييف" (Example: Problem with the air conditioning). Below the title field, there is a section labeled "الوصف" (Description) with a larger text input field containing the placeholder "اشرح المشكلة أو الاقتراح بالتفصيل..." (Explain the problem or suggestion in detail...). At the bottom of the form, there is a blue button labeled "إرسال" (Send) with a right-pointing arrow icon.

الشكل (14-4): شاشة شكوى أو اقتراح جديد

نموذج تقديم شكوى أو اقتراح جديد، مع إمكانية إرفاق صور توضيحية واختيار الإرسال بشكل مجهول.

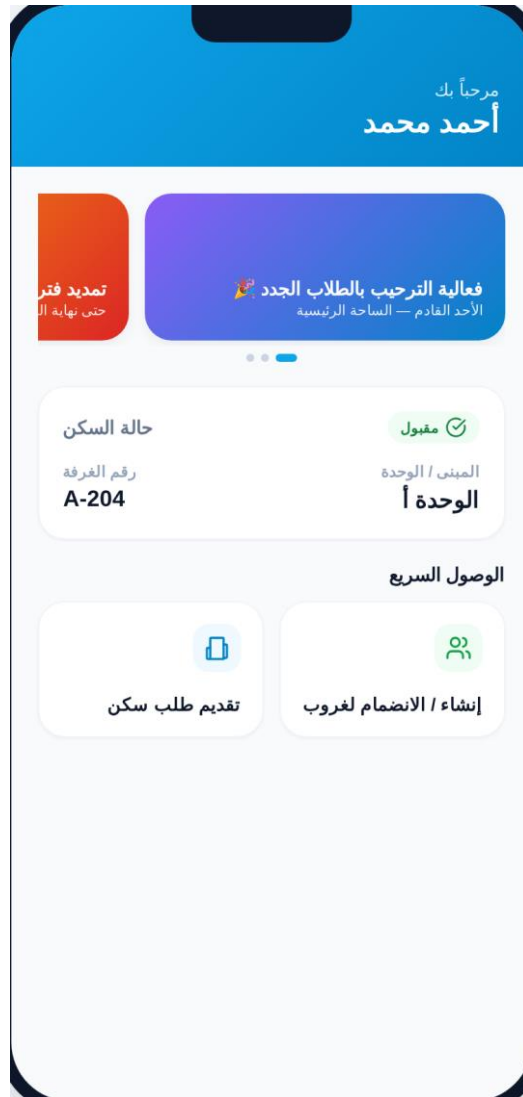
15. الغروبات: تأكيد قبول دعوة (Dialog) وتعطيل الدعوة عند الاكتمال



الشكل (15-4): شاشة الغروبات: تأكيد قبول دعوة (Dialog) وتعطيل الدعوة عند الاكتمال

نافذة منبثقة (Dialog) لتأكيد قبول دعوة الانضمام إلى مجموعة سكن، مع تعطيل الدعوة تلقائيًا عند اكتمال عدد أعضاء المجموعة.

16. الرئيسية (محدّثة: Swiper إعلانات الإدارة)



الشكل (4-16): شاشة الرئيسية (محدّثة: Swiper إعلانات الإدارة)

نسخة محدّثة من الصفحة الرئيسية تتضمن عارض إعلانات دّوار (Swiper) لعرض إعلانات الإدارة الموجهة للطلاب.

17. تعديل البيانات الشخصية

تعديل البيانات >

أ
تغيير الصورة الشخصية

البيانات القابلة للتعديل

الاسم الأول
أحمد

الاسم الأخير
محمد

رقم الجوال
05xxxxxxx

البريد الإلكتروني
ahmed@university.edu

مكان الإقامة
الرياض

بيانات ثابتة (غير قابلة للتعديل)

الرقم الجامعي
442100215

للتعديل تواصل مع إدارة السكن

الرقم الوطني / الإقامة

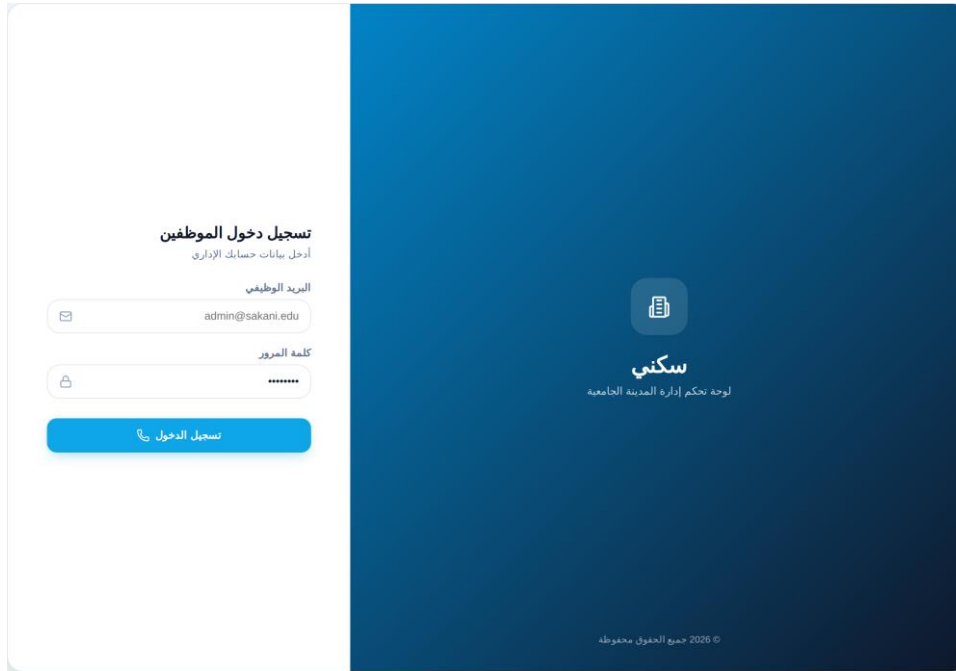
الشكل (17-4): شاشة تعديل البيانات الشخصية

شاشة تعديل البيانات الشخصية للطالب، وتميّز بوضوح بين الحقول القابلة للتعديل مباشرة (كالاسم ورقم الجوال) والحقول الثابتة التي تتطلب مراجعة إدارة السكن لتعديلها (كالرقم الجامعي والرقم الوطني).

ثانياً: لوحة تحكم الإدارة (React)

تضم لوحة التحكم 13 شاشة موجهة لفريق الإدارة، تغطي إدارة السكن والخدمات والتواصل مع الطلاب والصلاحيات:

1. تسجيل دخول الإدارة



الشكل (18-4): شاشة تسجيل دخول الإدارة

شاشة تسجيل دخول الإدارة إلى لوحة التحكم، وتستخدم من قبل المدير أو المدير العام.

2. نظرة عامة



الشكل (4-19): شاشة نظرة عامة

شاشة النظرة العامة (Overview)، وتعرض مؤشرات الأداء الرئيسية (KPIs) كعدد الشكاوى المفتوحة وطلبات السكن المعلقة ونسبة الإشغال، إلى جانب رسم بياني لإشغال المباني خلال الأسبوع.

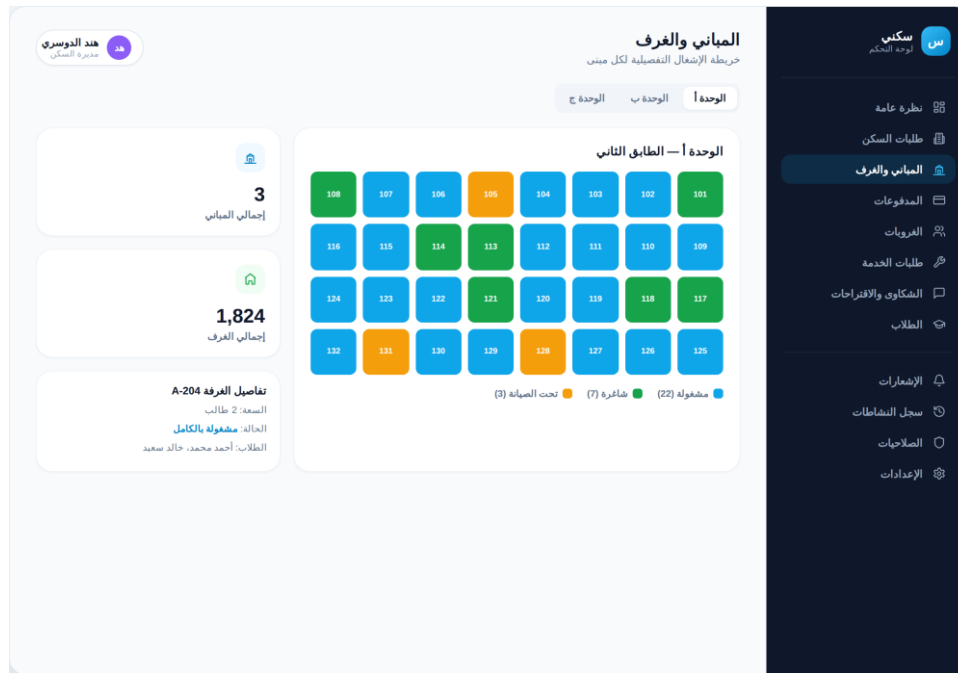
3. إدارة طلبات السكن

الطالب	التفصيل	تاريخ الطلب	الحالة	إجراء
خالد سعيد المطيري 442100138	الوحدة ب — فردية	منذ ساعة	قيد المراجعة	...
عمر فهد العنبي 442100223	الوحدة أ — مشتركة	منذ 3 ساعات	مقبول	...
يوسف علي الزهراني 442100341	الوحدة ج — فردية	أمس	قيد المراجعة	...
محمد أحمد القحطاني 442100489	الوحدة أ — مشتركة	أمس	مرفوض	...
سلطان ناصر الشمري 442100513	الوحدة ب — مشتركة	منذ يومين	مقبول	...
فيصل تركي الخري 442100651	الوحدة ج — فردية	منذ 3 أيام	مقبول	...

الشكل (4-20): شاشة إدارة طلبات السكن

شاشة إدارة طلبات السكن، تعرض جدولاً بجميع الطلبات وحالاتها (قيد المراجعة، مقبول، مرفوض) مع إمكانية اتخاذ إجراء على كل طلب.

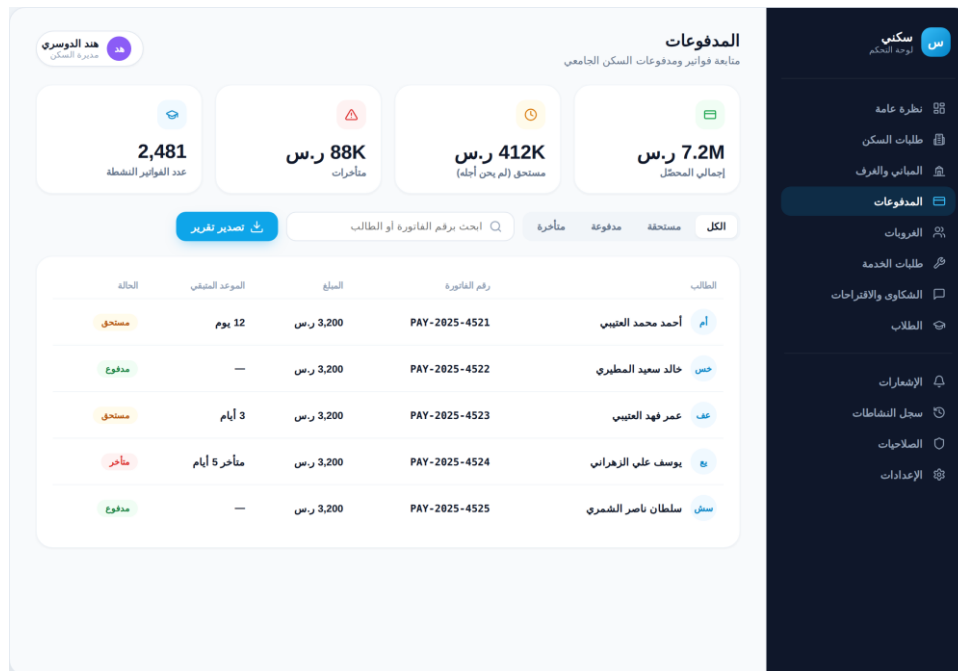
4. إدارة المباني والغرف



الشكل (4-21): شاشة إدارة المباني والغرف

شاشة إدارة المباني والغرف، وتعرض خريطة إشغال بصرية للوحدات السكنية والغرف وحالتها (مشغولة/شاغرة).

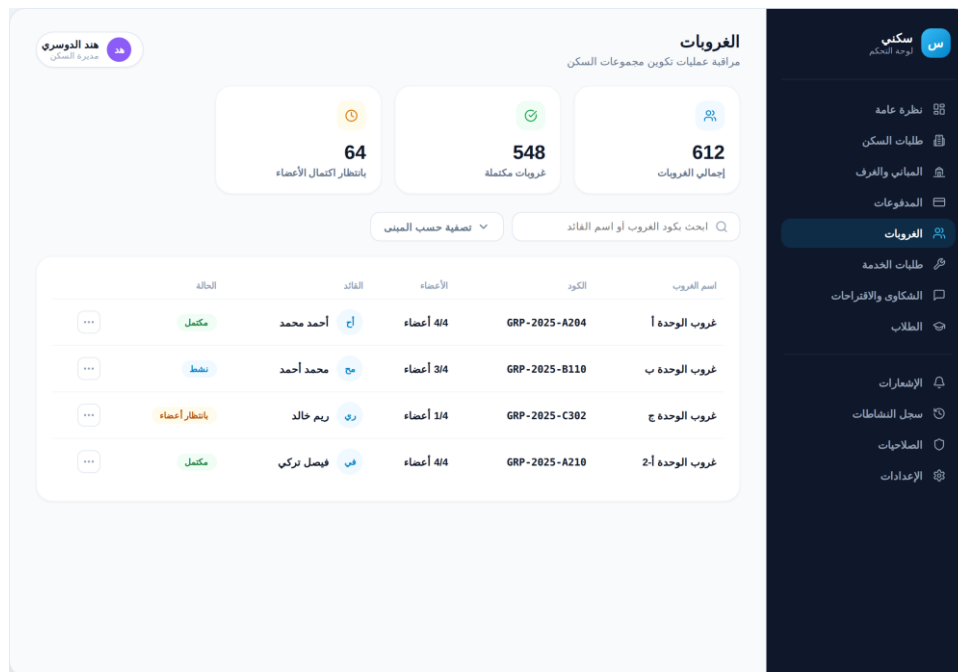
5. إدارة المدفوعات



الشكل (4-22): شاشة إدارة المدفوعات

شاشة إدارة المدفوعات المرتبطة برسوم السكن والخدمات.

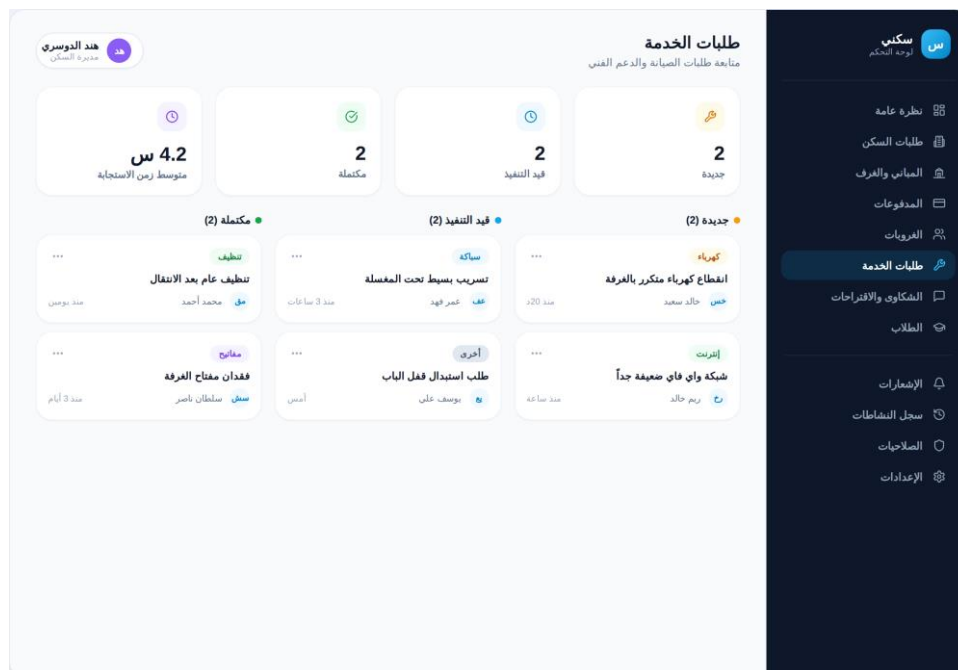
6. إدارة الغروبات



الشكل (4-23): شاشة إدارة الغروبات

شاشة إدارة مجموعات السكن (الغروبات)، وتتيح للإدارة متابعة تشكيل المجموعات والتدخل عند الحاجة.

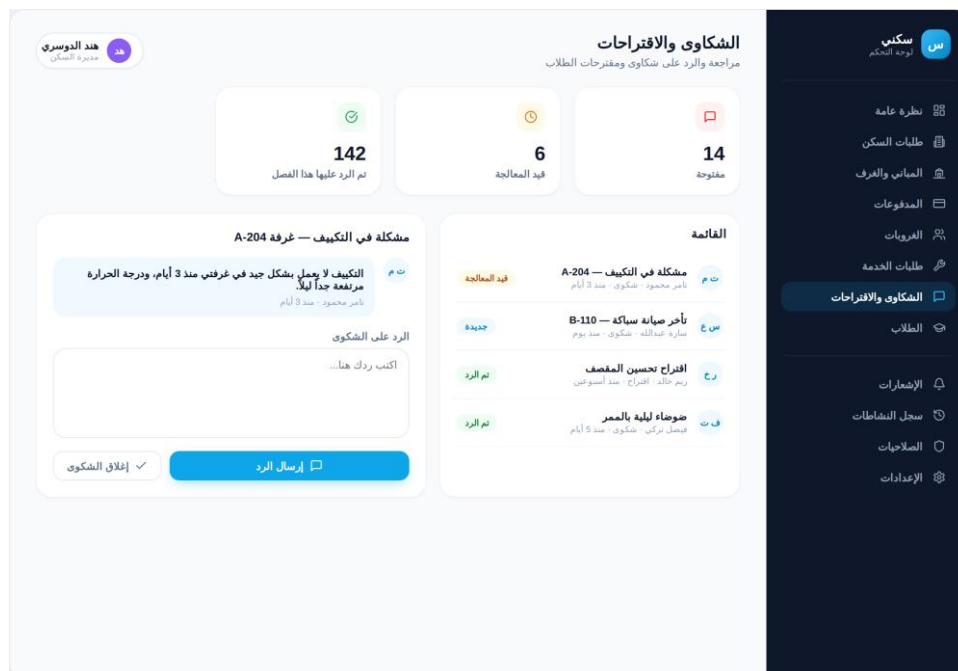
7. طلبات الخدمة



الشكل (4-24): شاشة طلبات الخدمة

لوحة Kanban لطلبات الخدمة، تعرض الطلبات موزّعة على أعمدة حسب حالتها (قيد الانتظار، قيد التنفيذ، منتهية) لتسهيل المتابعة التشغيلية.

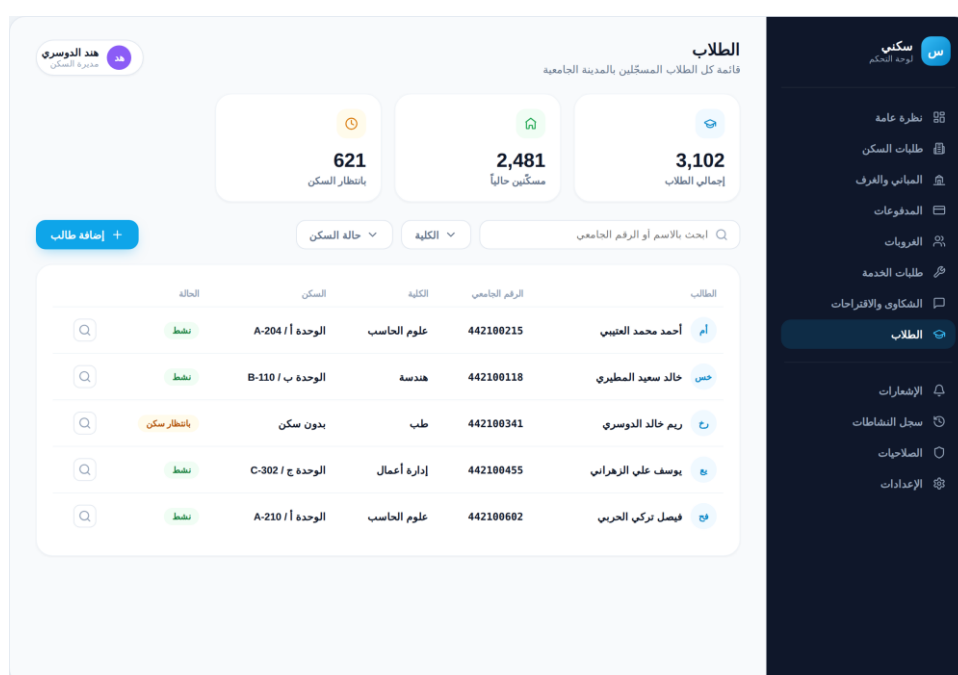
8. الشكاوى والاقتراحات



الشكل (25-4): شاشة الشكاوى والاقتراحات

شاشة الشكاوى والاقتراحات من جهة الإدارة، مع لوحة جانبية للرد المباشر على كل شكوى وتحديد حالتها كمقروءة.

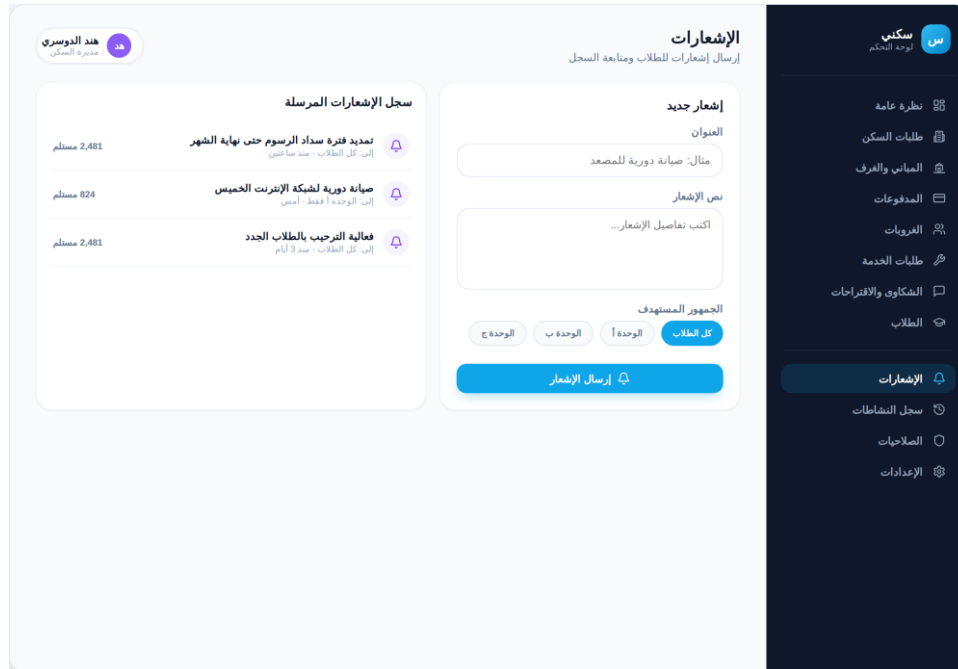
9. إدارة الطلاب



الشكل (4-26): شاشة إدارة الطلاب

شاشة قائمة الطلاب المسجلين في النظام، مع إمكانية البحث والتصفية والاطلاع على بيانات كل طالب.

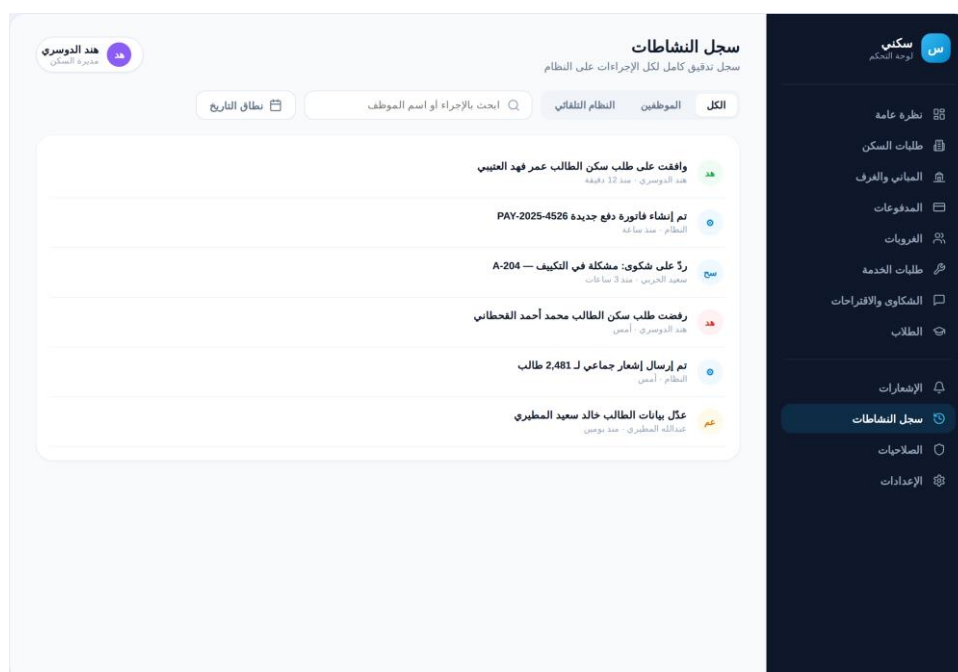
10. الإشعارات



الشكل (4-27): شاشة الإشعارات

شاشة إرسال الإشعارات، تتيح للإدارة إرسال إشعار إلى طالب محدد أو مجموعة من الطلاب أو جميعهم.

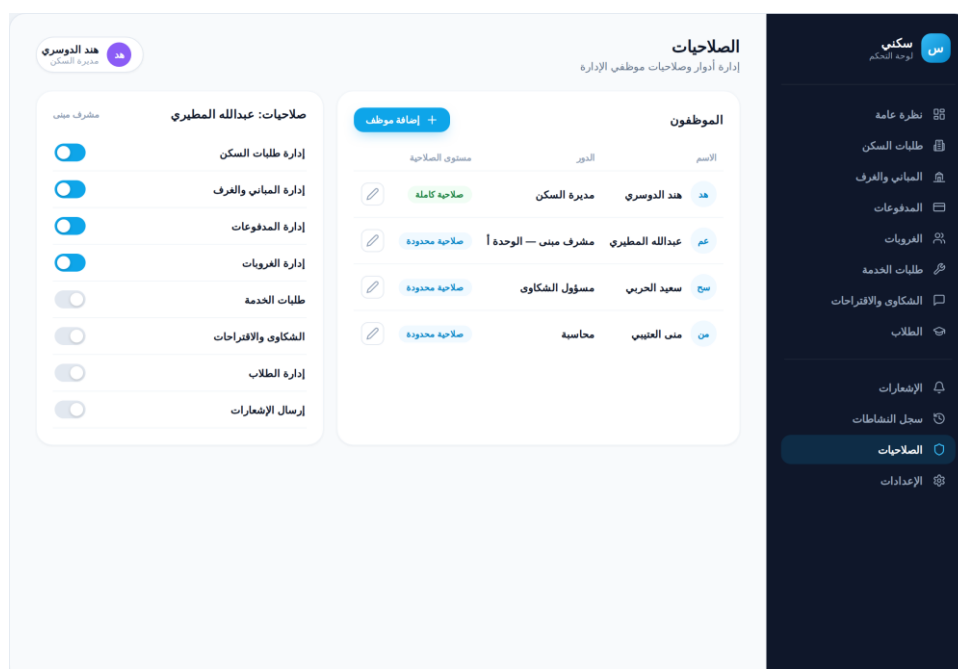
11. سجل النشاطات



الشكل (4-28): شاشة سجل النشاطات

شاشة سجل النشاطات (Activity Log)، وتوثق العمليات الإدارية الهامة التي جرت ضمن النظام لأغراض التتبع والمراجعة.

12. الصلاحيات



الشكل (4-29): شاشة الصلاحيات

شاشة الصلاحيات، وتتيح للمدير العام إدارة أدوار المستخدمين وتحديد صلاحيات كل دور ضمن لوحة التحكم.

13. الإعدادات العامة

الإعدادات العامة
إعدادات النظام والفصل الدراسي

إعدادات المدفوعات
قيمة الرسوم الفصلية (رسم) 3200
مهلة السداد (أيام) 14
تفعيل تذكير تلقائي بالدفع ☒

إعدادات الفصل الدراسي
الفصل الدراسي الحالي
الفصل الثاني 2025/2026
موعد بدء التسيكين 2026/01/15
الحد الأقصى لأعضاء القروب 4

النسخ الاحتياطي
آخر نسخة احتياطية: منذ 6 ساعات
يتم أخذ نسخة تلقائية يومياً الساعة 3:00 صباحاً
إنشاء نسخة احتياطية الآن

إعدادات الإشعارات
إشعار تلقائي عند تحديث حالة الطلب ☒
إشعار عند فتح شكوى جديدة ☒

سكني
لوحة التحكم

- نظرة عامة
- طلبات السكن
- المباني والغرف
- المدفوعات
- القروض
- طلبات الخدمة
- الشكاوى والاقتراحات
- الطلاب
- الإشعارات
- سجل النشاطات
- الصلاحيات
- الإعدادات

الشكل (4-30): شاشة الإعدادات العامة

شاشة الإعدادات العامة للنظام، وتشمل الإعدادات المؤسسية العامة لمنصة سكني.

- .Microsoft, "ASP.NET Core Documentation," Microsoft Docs [1]
- .Microsoft, "Entity Framework Core Documentation," Microsoft Docs [2]
- .MongoDB Inc., "MongoDB Manual," MongoDB Documentation [3]
- ".OpenJS Foundation, "Node.js Documentation [4]
- ".Meta / OpenJS, "React Documentation [5]
- ".Google, "Flutter Documentation [6]
- ".Docker Inc., "Docker Documentation [7]
- ".ThreeMammals, "Ocelot API Gateway Documentation [8]
- ".draw.io / diagrams.net, "Diagramming Documentation [9]

English Abstract

This thesis documents "Sakani", an integrated software platform for managing university housing affairs, built on a microservices architecture with Docker containerization. The system is composed of five independent services — Authentication, Housing, Service Requests, Feedback & Suggestions, and Notifications & Announcements — communicating through a unified Ocelot API Gateway.

The platform serves students through a Flutter mobile application and administrators through a React web dashboard, both consuming the same REST API contracts. The Authentication service uses Node.js, Express, and MongoDB, issuing JWT access tokens together with refresh tokens, and stores only hashed passwords. The remaining services are built with ASP.NET Core and SQL Server, each owning an independent database in line with the database-per-service principle.

This document presents the system's conceptual and per-service Entity-Relationship Diagrams, Use Case diagrams, the technology choices and their engineering justification, and the platform's user interface design. It also records a set of deliberate design decisions — most notably converting the user-to-service-request relationship into a proper junction table to preserve request history while enforcing a single-active-request business rule in application logic rather than in the database schema.